

The University of Faisalabad

LAB MANUAL

SUBMITTED TO: SIR ARHAM

SUBMITTED BY: EMAN TAHIR

ROLL NO. 2023-BS-AI-015

SUBJECT: DATA MINING

DEPARTMENT: ARTIFICIAL INTELLIGENCE

Contents

LAB 1	3
LAB 2	11
LAB 3	20
LAB 4	27
LAB 5	34
LAB 7	41
LAB 8	45
LAB 9	49
LAB 10	54
LAB 11	58
LAB 12	62
LAB 13	66
LAB 14	70
LAB 15	84

LAB 1

INTRODUCTION TO DATA MINING SOFTWARE & DATASETS

Definition: Data mining is the process of discovering meaningful patterns, correlations, and insights from large datasets using statistical, machine learning, and database techniques.

Purpose: Helps in decision-making, prediction, and knowledge discovery.

Applications: Business intelligence, healthcare, fraud detection, recommendation systems, scientific research.

Common Data Mining Software

Software	Key Features	Use Cases
WEKA	Open-source, GUI-based, supports classification, clustering, association rules	Academic learning, prototyping
RapidMiner	Drag-and-drop interface, supports advanced ML workflows	Industry applications, predictive analytics
Orange	Visual programming, interactive data visualization	Teaching, exploratory analysis
R / Python (Scikit-learn, Pandas)	Flexible, script-based, supports advanced ML algorithms	Research, production-level systems
SQL-based Tools	Query-driven analysis	Database mining

Types of Datasets in Data Mining

Dataset Categories

- **Structured Data:** Tabular format (rows & columns), e.g., sales records, student grades.
- **Unstructured Data:** Text, images, audio, video.
- **Semi-structured Data:** JSON, XML logs.
- **Transactional Data:** Market basket analysis, purchase histories.
- **Time-Series Data:** Stock prices, sensor readings.
- **Spatial Data:** Maps, geographic information.
- **Web & Social Media Data:** Clickstreams, tweets, user interactions.

Sources of Datasets

- **Public Repositories:** UCI Machine Learning Repository, Kaggle, OpenML.
- **Government Portals:** Data.gov, WHO datasets.
- **Enterprise Databases:** CRM, ERP systems.
- **Generated Data:** Simulated datasets for testing algorithms.

Example

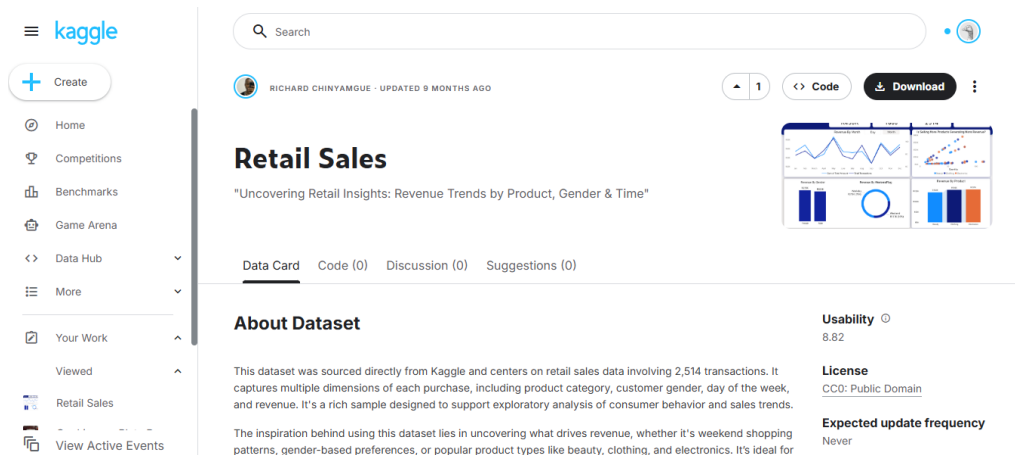
Dataset: Iris flower dataset (150 samples, 4 features, 3 classes).

Steps:

1. Load dataset in WEKA.
2. Explore attributes (sepal length, sepal width, petal length, petal width).
3. Visualize class distribution.
4. Apply a simple classifier (e.g., Decision Tree).
5. Observe accuracy and confusion matrix.

Lab Work

Dataset



Application



1. Create workflow

Create a new workflow

Workflow name

retail sales task-01

Cancel Create

2. Use CSV reader node



3. Load dataset

Dialog - 3:1 - CSV Reader

File

Settings Transformation Advanced Settings Limit Rows Encoding Flow Variables Job Manager Selection Memory Policy

Input location

Read from Local File System

Mode ☒ File ☐ Files in folder

File C:\Users\hp\OneDrive\Desktop\dm\retail_sales_dataset.csv Browse...

Reader options

Format

Autodetect format

Column delimiter , Row delimiter ☒ Line break ☐ Custom \n

Quote char " Quote escape char \"

Comment char #

☒ Has column header ☐ Has RowID

☐ Support short data rows ☐ Prepend file index to RowID

Preview

The suggested column types are based on the first 10000 rows only. See 'Advanced Settings' tab.

Row ID	I	Transac...	S	Date	S	Custom...	S	Gender	I	Age	S	Product...	I	Quantity	I	Price p...	I	Total A...
Row0	1			2023-11-24		CUST001		Male		34		Beauty		3		50		150
Row1	2			2023-02-27		CUST002		Female		26		Clothing		2		500		1000
Row2	3			2023-01-13		CUST003		Male		50		Electronics		1		30		30
Row3	4			2023-05-21		CUST004		Male		37		Clothing		1		500		500

OK Apply Cancel ?

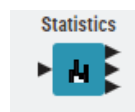
4. Verify Dataset

Rows: 1000 | Columns: 9

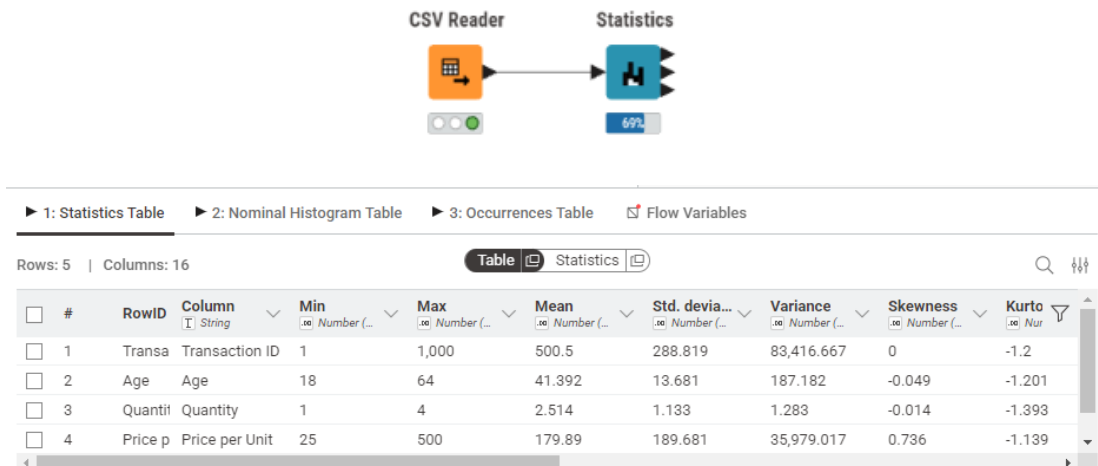
Table Statistics

#	RowID	Transacti...	Date	Customer...	Gender	Age	Product C...	Quantity	Price
1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50
2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500
3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30
4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500

5. Use statistics node



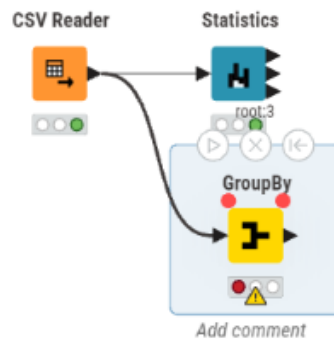
6. Connect and execute

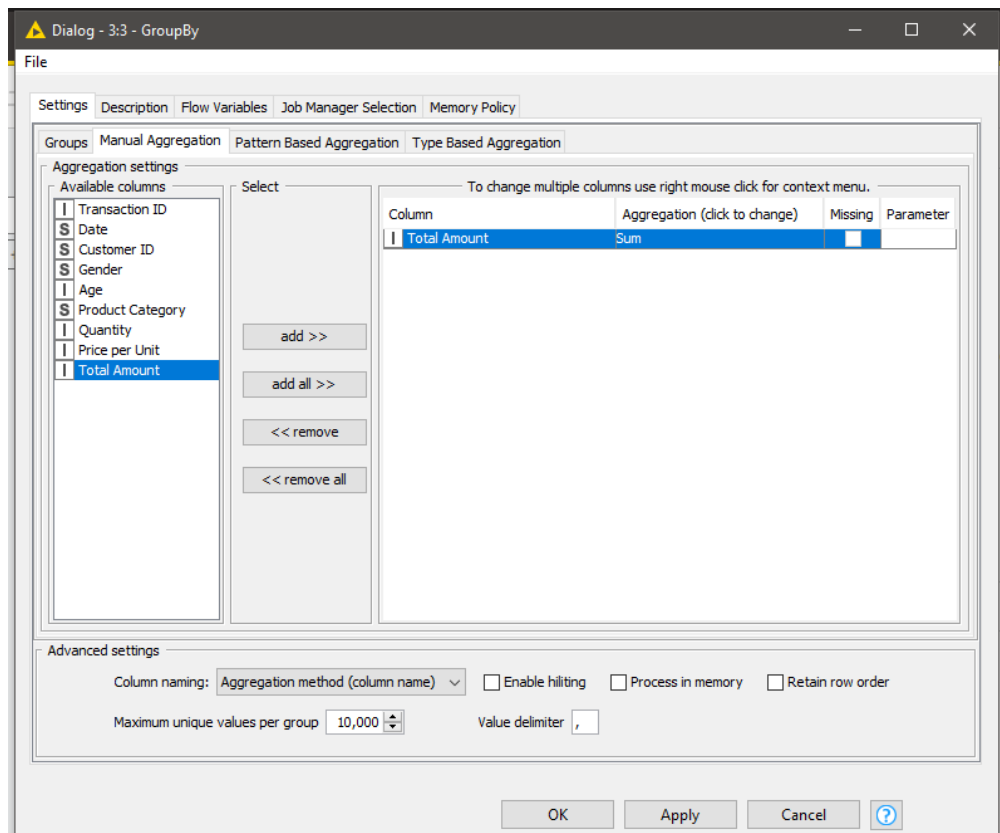


7. Using group-by node



8. KPI-1





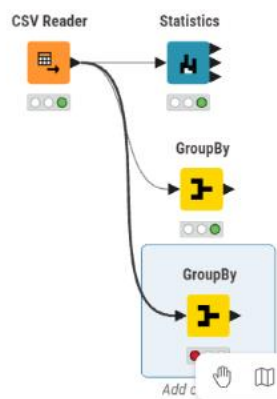
After execution

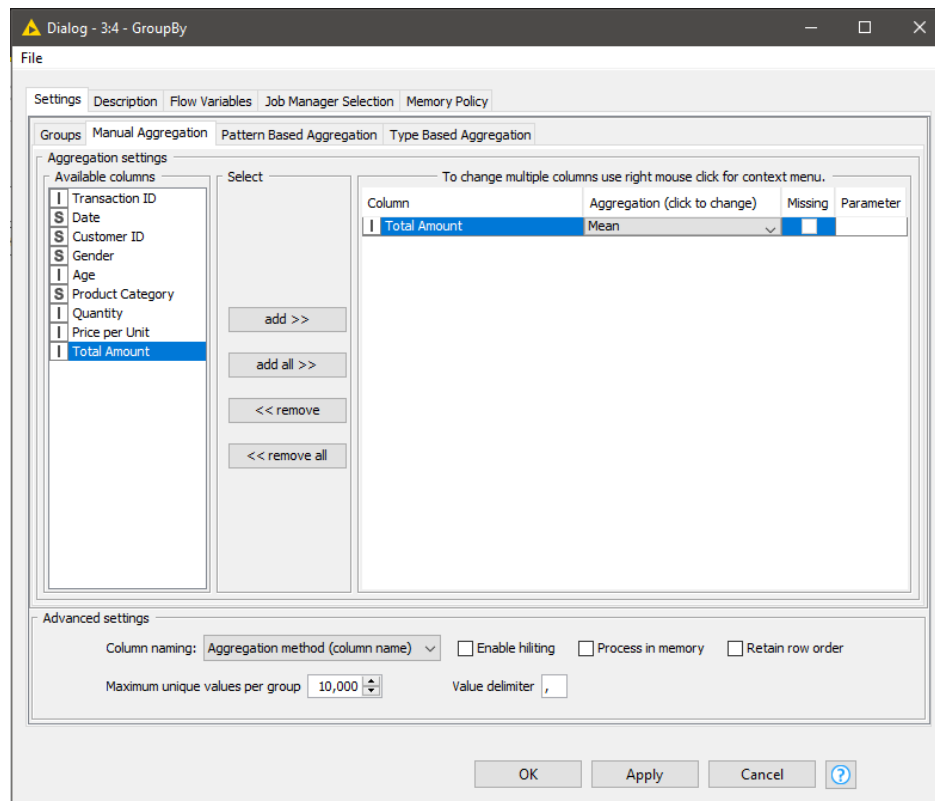
► 1: Group table Flow Variables

Rows: 1 | Columns: 1 Table Statistics

#	RowID	Sum(Total Amount)
1	Row0	456000

9. KPI-2





After Execution

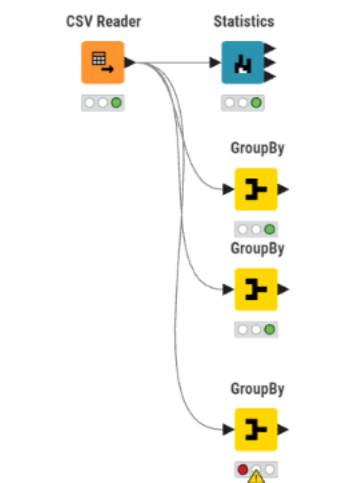
► 1: Group table ☒ Flow Variables

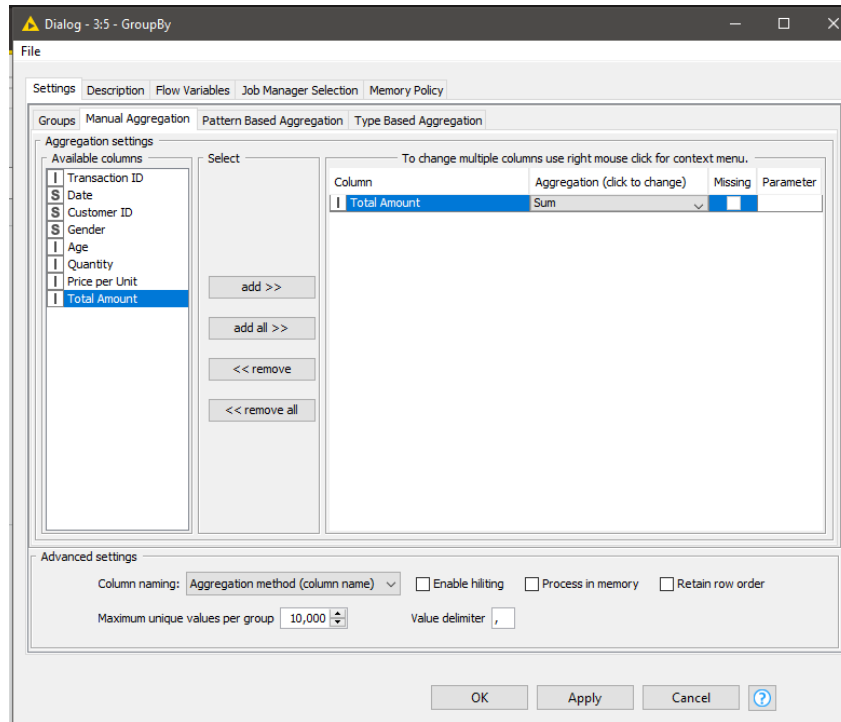
Rows: 1 | Columns: 1

Table ☒ Statistics ☐

#	RowID	Mean(Total Amount)
1	Row0	456

10. KPI-3





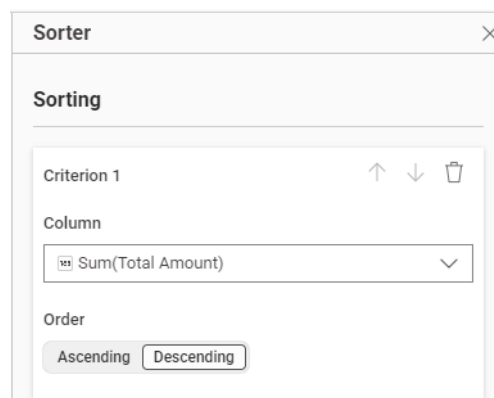
After Execution

► 1: Group table Flow Variables

Rows: 3 | Columns: 2 Table Statistics

#	RowID	Product Category	Sum(Total Amount)
1	Row0	Beauty	143515
2	Row1	Clothing	155580
3	Row2	Electronics	156905

11. Using sorter node



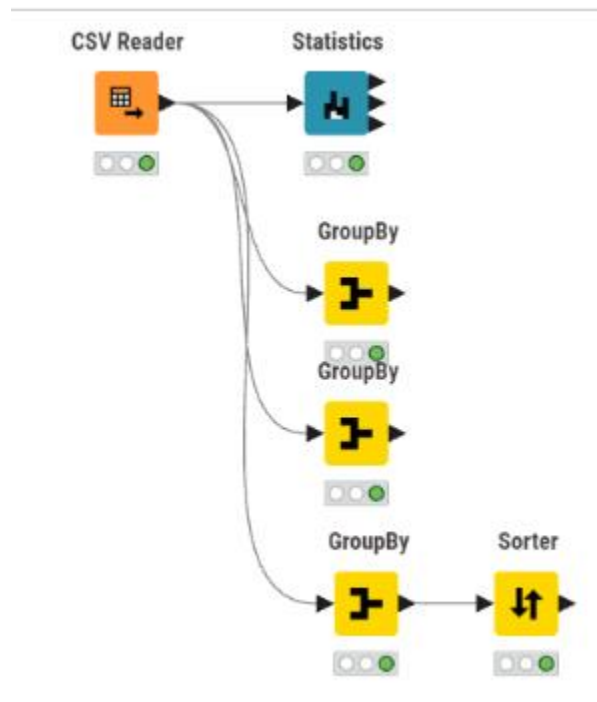
After Execution

► 1: Sorted Table Flow Variables

Rows: 3 | Columns: 2 Table Statistics Search Filter

☐	#	RowID	Product Category String	Sum(Total Amount) Number (Integer)	☐
☐	1	Row2	Electronics	156905	
☐	2	Row1	Clothing	155580	
☐	3	Row0	Beauty	143515	

12. Final Diagram



LAB 2

CRISP-DM PROCESS & DATA TYPES

Definition: CRISP-DM is the most widely used methodology for data mining projects.

Purpose: Provides a structured approach to planning, executing, and evaluating data mining tasks.

Phases: Six iterative stages that guide the entire process.

Phases of CRISP-DM

Phase	Description	Example Activity
1. Business Understanding	Define objectives, requirements, and success criteria.	Identify goal: predict customer churn.
2. Data Understanding	Collect, describe, and explore data.	Load dataset, check distributions, detect anomalies.
3. Data Preparation	Clean and transform data for modeling.	Handle missing values, normalize features.
4. Modeling	Apply algorithms to extract patterns.	Train decision tree, clustering, regression.
5. Evaluation	Assess model performance against business goals.	Compare accuracy, precision, recall.
6. Deployment	Implement results in real-world use.	Integrate churn prediction into CRM system.

Data Types in Data Mining

- **Numerical Data**

Continuous: Real numbers (e.g., temperature, salary).

Discrete: Countable values (e.g., number of purchases).

- **Categorical Data**

Nominal: Labels without order (e.g., gender, color).

Ordinal: Ordered categories (e.g., education level, rankings).

- **Text Data**

Unstructured, requires NLP techniques (e.g., reviews, tweets).

- **Time-Series Data**

Sequential observations over time (e.g., stock prices, weather).

- **Spatial Data**

Geographic or location-based (e.g., maps, GPS coordinates).

- **Multimedia Data**

Images, audio, video requiring specialized mining techniques.

Example

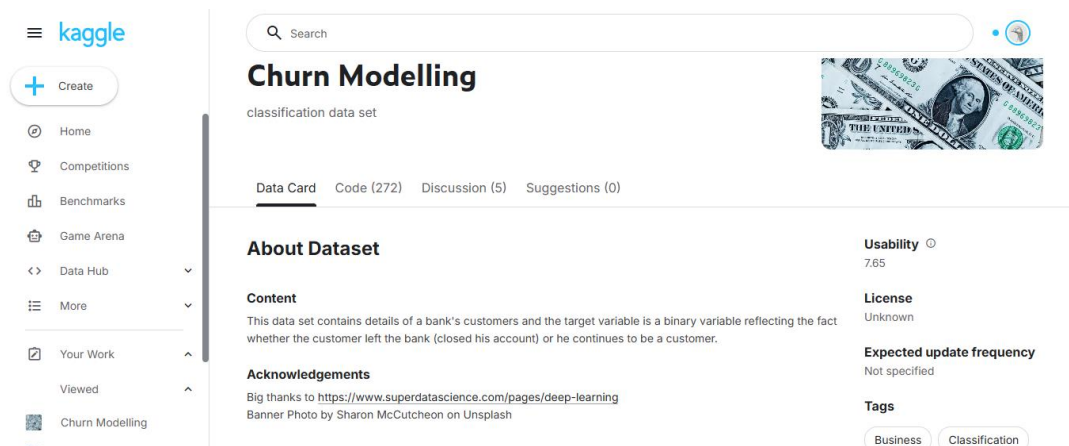
Dataset: Titanic survival dataset.

CRISP-DM Mapping:

1. Business Understanding: Predict survival of passengers.
2. Data Understanding: Explore age, gender, class attributes.
3. Data Preparation: Handle missing ages, encode categorical features.
4. Modeling: Train logistic regression classifier.
5. Evaluation: Assess accuracy and ROC curve.
6. Deployment: Use model to predict survival for new passengers.

Lab Work

Dataset



The screenshot shows the Kaggle interface for the 'Churn Modelling' dataset. On the left is a sidebar with navigation links: Home, Competitions, Benchmarks, Game Arena, Data Hub, More, Your Work, Viewed, and Churn Modelling. The main content area has a search bar at the top. Below it, the dataset title 'Churn Modelling' is displayed with a subtitle 'classification data set' and a thumbnail image of US dollar bills. A tab bar shows 'Data Card' (selected), 'Code (272)', 'Discussion (5)', and 'Suggestions (0)'. The 'About Dataset' section includes a 'Content' paragraph: 'This data set contains details of a bank's customers and the target variable is a binary variable reflecting the fact whether the customer left the bank (closed his account) or he continues to be a customer.' It also has an 'Acknowledgements' section with a link to a SuperDataScience article and a credit to Sharon McCutcheon. On the right, a 'Usability' score of 7.65 is shown, along with 'License' (Unknown), 'Expected update frequency' (Not specified), and 'Tags' (Business, Classification).

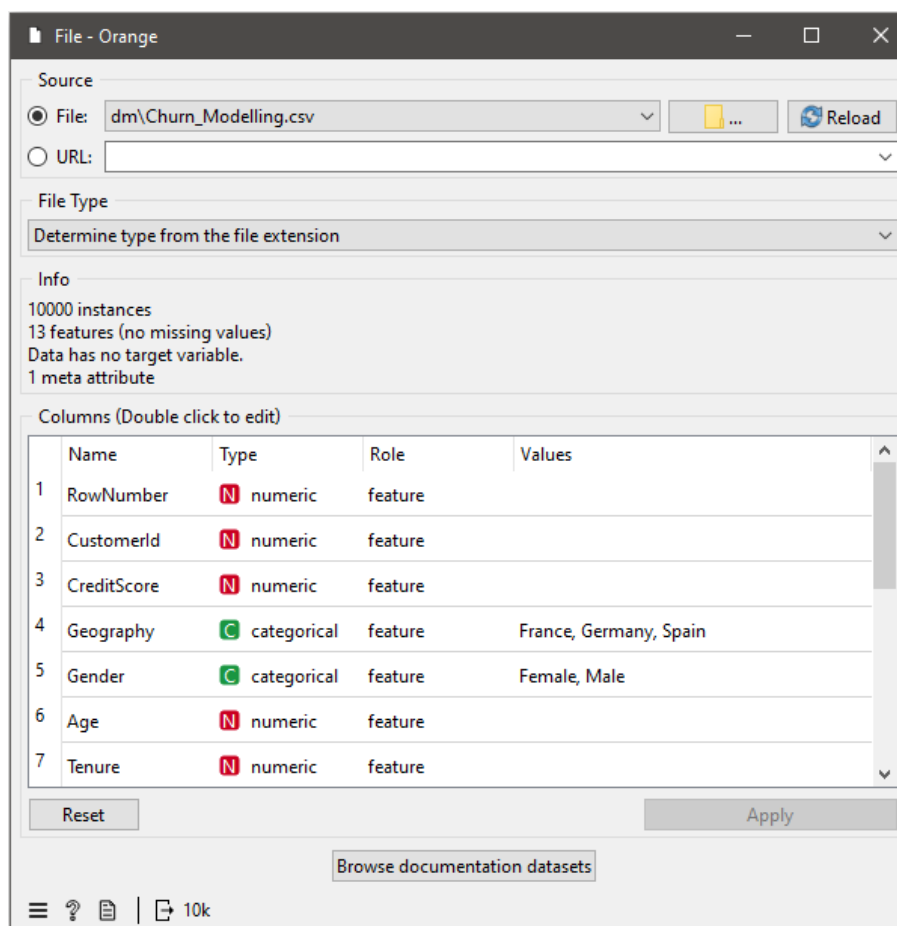
Application



1. Use file and data table widget



2. Load dataset

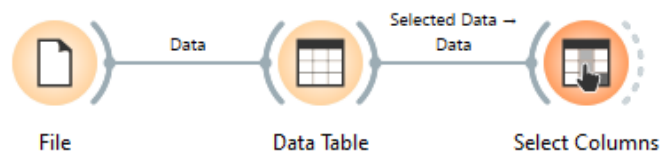


3. Check dataset

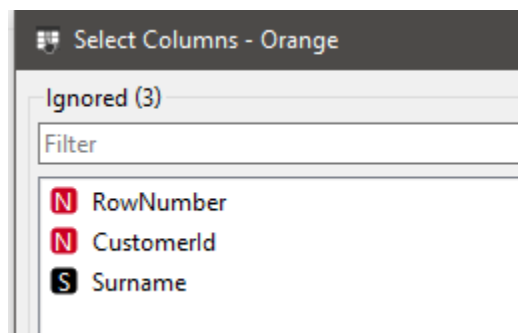
The screenshot shows the 'Data Table - Orange' window. On the left, the 'Info' panel displays: 10000 instances (no missing data), 13 features, No target variable, 1 meta attribute. The 'Variables' panel has checkboxes for 'Show variable labels (if present)' (checked), 'Visualize numeric values' (unchecked), and 'Color by instance classes' (checked). The 'Selection' panel has a 'Clear Selection' button and a checked 'Select full rows' option. The 'Restore Original Order' button is also visible. The main table displays the following data:

	Surname	RowNumber	CustomerId	CreditScore	Geography
1	Hargrave	1	15634602	619	France
2	Hill	2	15647311	608	Spain
3	Onio	3	15619304	502	France
4	Boni	4	15701354	699	France
5	Mitchell	5	15737888	850	Spain
6	Chu	6	15574012	645	Spain
7	Bartlett	7	15592531	822	France
8	Obinna	8	15656148	376	Germany
9	He	9	15792365	501	France
10	H?	10	15592389	684	France
11	Bearce	11	15767821	528	France
12	Andrews	12	15737173	497	Spain
13	Kay	13	15632264	476	France
14	Chin	14	15691483	549	France
15	Scott	15	15600882	635	Spain
16	Goforth	16	15643966	616	Germany
17	Romeo	17	15737452	653	Germany
18	Henderson	18	15788218	549	Spain

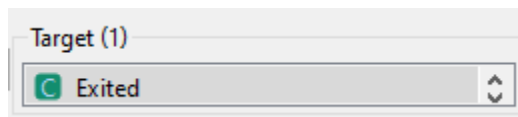
4. Adding selected columns



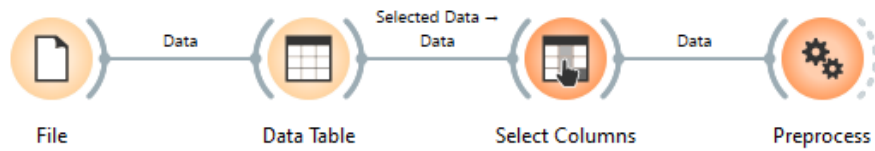
5. Move RowNumber, CustomerId and Surname into ignore list



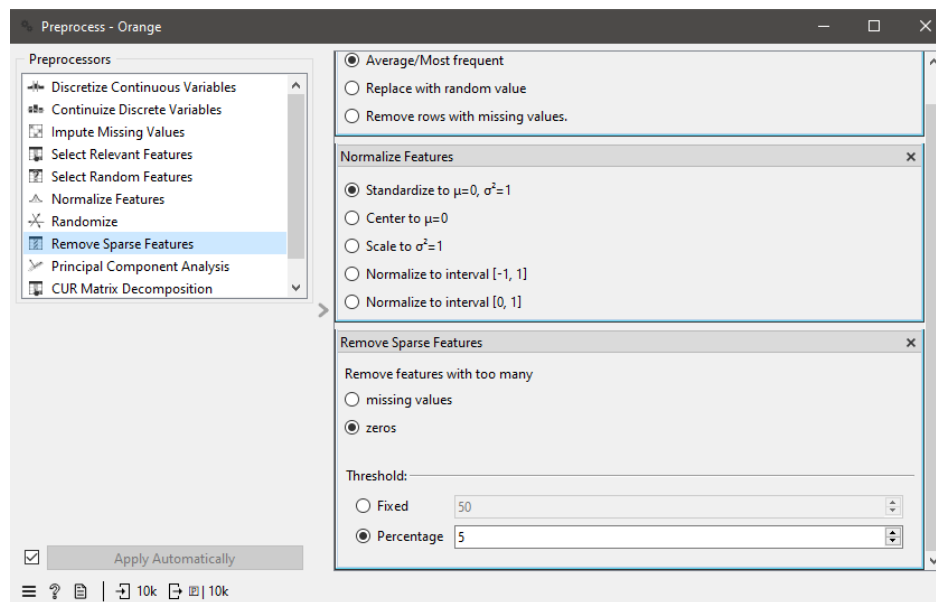
6. Move Exited to Target



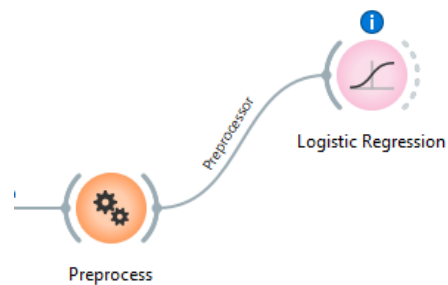
7. Connect preprocess node



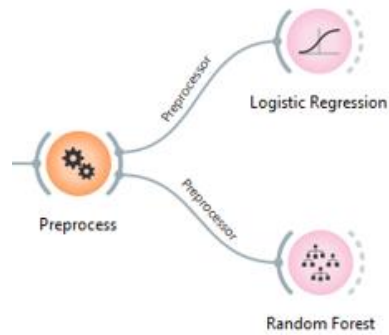
8. Normalize Min-Max



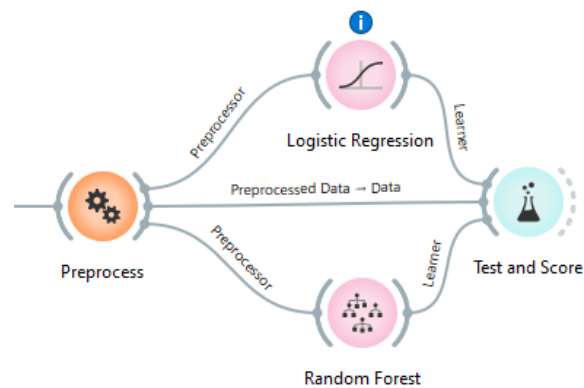
9. Add logistic regression node



10. And random forest node



11. Connect all 3 to test score node



Inside test score

Test and Score - Orange

☒ Cross validation

Number of folds: 10

☒ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 10

Training set size: 66 %

☒ Stratified

☐ Leave one out

☐ Test on train data

☐ Test on test data

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.735	0.779	0.717	0.698	0.779	0.049
Random Forest	0.792	0.830	0.816	0.814	0.830	0.412

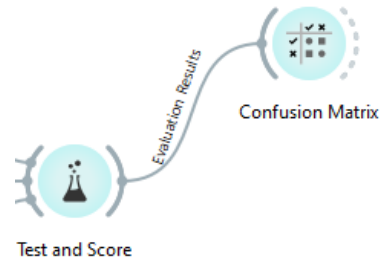
Compare models by: Area under ROC curve

☐ Negligible diff.: 0.1

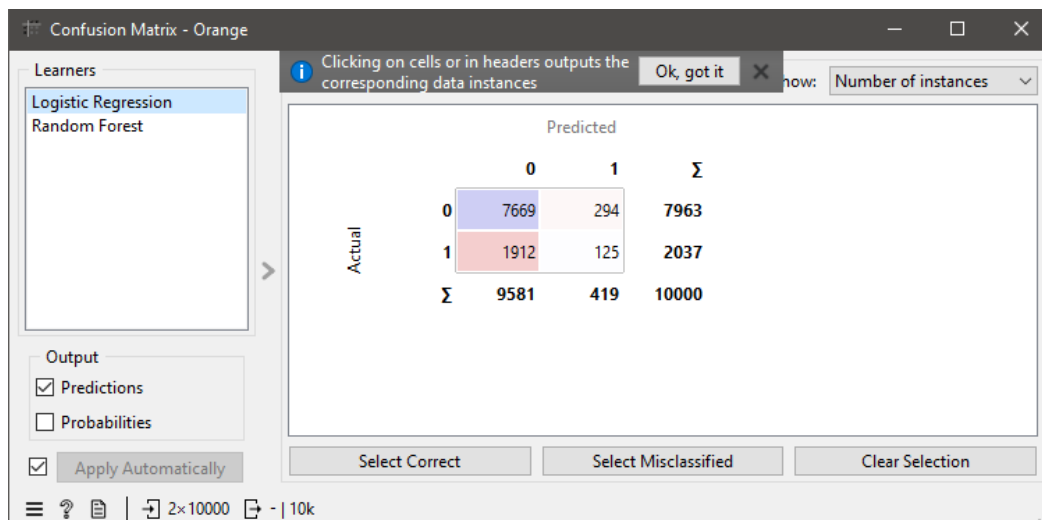
	Logistic Reg...	Random Fo...
Logistic Regression		0.000
Random Forest	1.000	

Table shows probabilities that the score for the model in the row is higher than that of the model in the column. Small numbers show the probability that the difference is negligible.

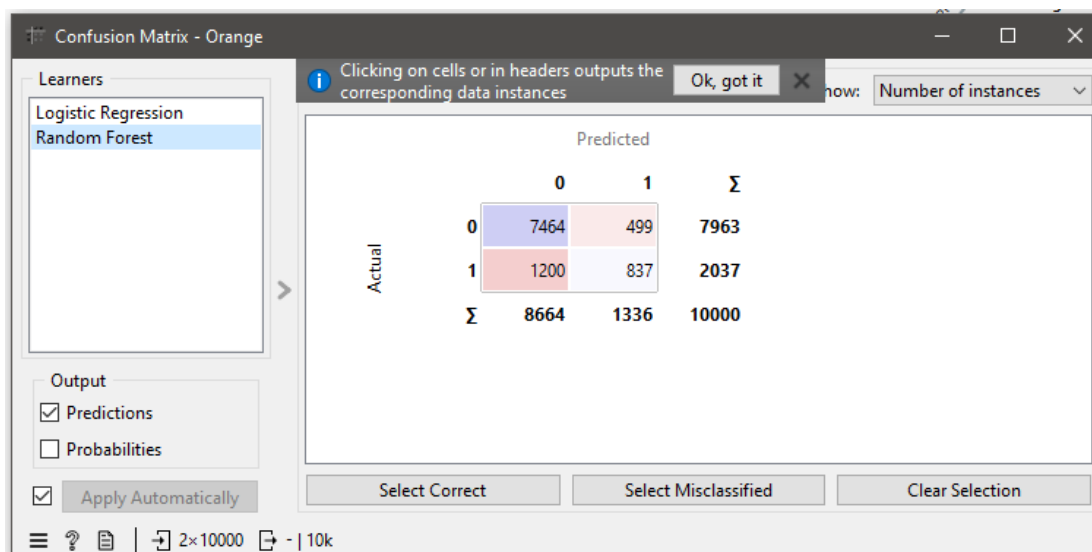
12. Add confusion matrix to test score



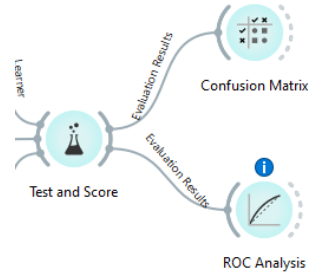
For logistic regression



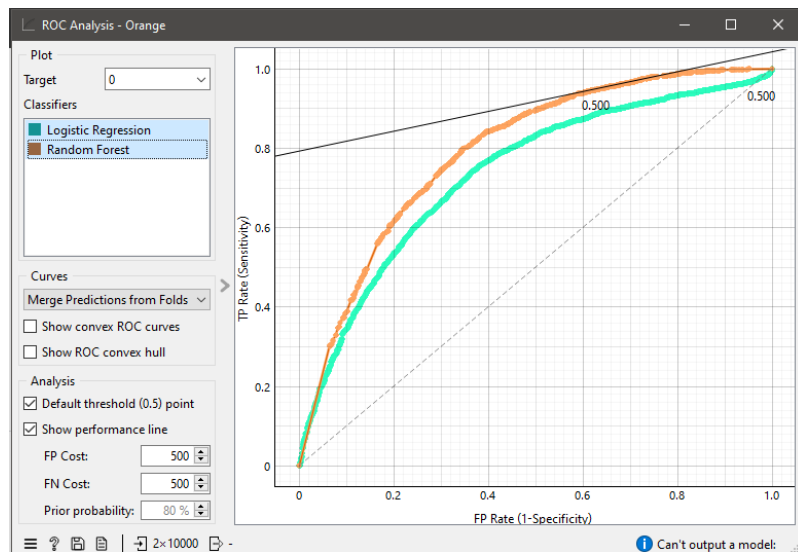
For random forest



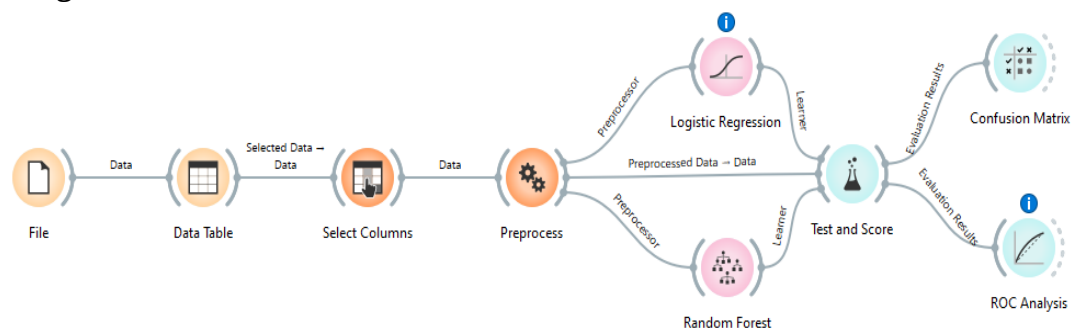
13. And ROC analysis



From ROC analysis



14. Final Diagram



Churn Rate	= 20.37%
Retention Rate	= 79.63%
Logistic Regression AUC	= 0.735
Random Forest AUC	= 0.792
Accuracy	= 82.8%

LAB 3

DATA CLEANING TECHNIQUES

Importance

- **Improves Accuracy:** Clean data ensures reliable results.
- **Reduces Bias:** Removes inconsistencies that can distort models.
- **Enhances Efficiency:** Preprocessed data speeds up mining algorithms.
- **Supports Better Decisions:** Clean datasets lead to trustworthy insights.

Common Data Quality Issues

- **Missing Values:** Empty cells or incomplete records.
- **Noisy Data:** Random errors, outliers, or irrelevant values.
- **Inconsistent Data:** Conflicting formats or duplicated entries.
- **Redundant Data:** Duplicate records or unnecessary attributes.
- **Incorrect Data Types:** Numbers stored as text; categorical values misclassified.

Data Cleaning Techniques

1. Handling Missing Data

Deletion: Remove rows/columns with too many missing values.

Imputation: Fill missing values using mean, median, mode, or predictive models.

Interpolation: Estimate missing values based on trends (time-series).

2. Handling Noisy Data

Binning: Group values into bins and smooth them.

Regression: Fit a regression model to smooth data.

Clustering: Detect and remove outliers.

3. Handling Inconsistent Data

Standardization: Convert formats (e.g., date formats, units).

Validation Rules: Apply constraints to ensure consistency.

Deduplication: Remove duplicate records.

4. Handling Redundant Data

Attribute Reduction: Remove irrelevant features.

Record Deduplication: Merge or delete duplicate entries.

Example

Dataset: Titanic survival dataset.

Issues: Missing ages, duplicate passenger records, inconsistent cabin entries.

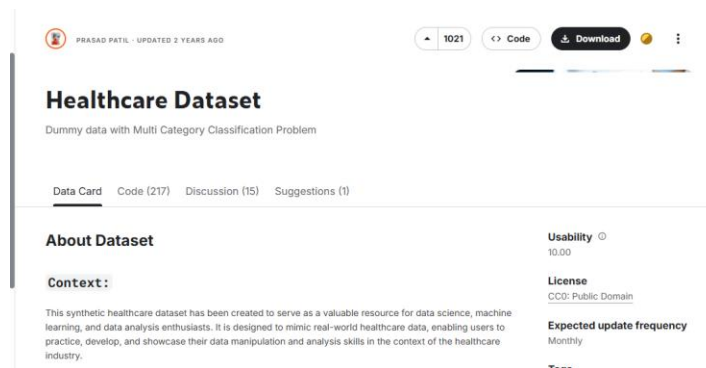
Cleaning Steps:

1. Impute missing ages with median.
2. Remove duplicate passenger IDs.
3. Standardize categorical variables (e.g., "male"/"Male").

Result: A clean dataset ready for modeling survival prediction.

Lab Work

Dataset



1. Import libraries

[1]:

```
import pandas as pd
import numpy as np
```

2. Load dataset

[21]:

```
try:
    df = pd.read_csv('healthcare_dataset.csv')
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("File not found. Generating a mock dataset to demonstrate the cleaning process.")
    # Generating mock data matching the dataset structure for standalone execution
    np.random.seed(42)
    mock_data = {
        'Patient_ID': [101, 102, 103, 104, 101, 105, 106, 107],
        'Name': ['Tiffany Ramirez', 'Ruben Burns', 'Chad Byrd', 'Antonio Frederick', 'Tiffany Ramirez', 'Ruben Burns', 'Chad Byrd', 'Antonio Frederick'],
        'Age': [81, -5, 61, 49, 81, 51, 41, 300], # Contains noise (-5, 300)
        'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Male', 'Male'],
        'Billing Amount': [37490.98, 47304.06, np.nan, 23303.32, 37490.98, 18086.06, 37490.98, 18086.06]
    }
    df = pd.DataFrame(mock_data)
```

Dataset loaded successfully!

3. Record initial counts for KPI calculations

```
[23]:
initial_rows = len(df)
initial_missing = df.isnull().sum().sum()
initial_duplicates = df.duplicated().sum()

print(f"\n--- Initial State ---")
print(f"Total Records: {initial_rows}")
print(f"Total Missing Values: {initial_missing}")
print(f"Total Duplicate Rows: {initial_duplicates}")
```

```
--- Initial State ---
Total Records: 55500
Total Missing Values: 0
Total Duplicate Rows: 534
```

4. Handle missing values

```
[25]:
if 'Gender' in df.columns:
    df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

# For numerical columns (like Billing Amount), we fill with the Median
if 'Billing Amount' in df.columns:
    df['Billing Amount'] = df['Billing Amount'].fillna(df['Billing Amount'].median())
```

5. Remove duplicates

```
[27]:
df_cleaned = df.drop_duplicates().reset_index(drop=True)
```

```
[28]:
```

6. Noise filtering

```
[29]:
if 'Age' in df_cleaned.columns:
    df_cleaned = df_cleaned[(df_cleaned['Age'] >= 0) & (df_cleaned['Age'] <= 120)]
```

7. Evaluation metrics

```
[31]:
final_rows = len(df_cleaned)
final_missing = df_cleaned.isnull().sum().sum()
removed_duplicates = initial_duplicates - df_cleaned.duplicated().sum()
```

8. Calculate metrics

[33]:

```
missing_reduced_pct = ((initial_missing - final_missing) / initial_missing * 100)
duplicate_removal_pct = (initial_duplicates / initial_rows * 100) if initial_rows
```

9. Cleaned KPIs

[35]:

```
missing_penalty = (final_missing / (final_rows * len(df_cleaned.columns))) * 50
data_quality_score = max(0, 100 - missing_penalty)

print(f"\nCleaned KPIs")
print(f"1. % Missing Reduced: {missing_reduced_pct:.2f}%")
print(f"2. Duplicate Removal Impact (% of Initial Data): {duplicate_removal_pct:.2f}%")
print(f"3. Final Cleaned Rows Remaining: {final_rows} (Dropped {initial_rows - final_rows})")
print(f"4. Estimated Data Quality Score: {data_quality_score:.2f}/100")
```

Cleaned KPIs

1. % Missing Reduced: 100.00%
2. Duplicate Removal Impact (% of Initial Data): 0.96%
3. Final Cleaned Rows Remaining: 54966 (Dropped 534 noisy/duplicate rows)
4. Estimated Data Quality Score: 100.00/100

10. Cleaned data

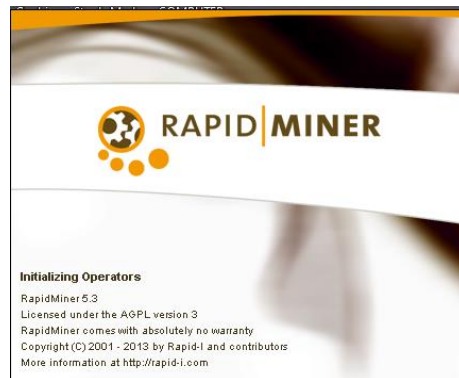
[37]:

```
df_cleaned.to_csv('cleaned_healthcare_dataset.csv', index=False)
print("\nCleaned dataset exported to 'cleaned_healthcare_dataset.csv'")
```

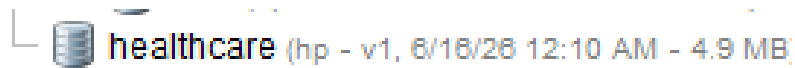
Cleaned dataset exported to 'cleaned_healthcare_dataset.csv'

	Name	Age	Gender	Blood Type
1	Bobby JacksOn	30	Male	
2	LesLie TErRy	62	Male	
3	DaNnY sMiTh	76	Female	
4	andrEw waTIS	28	Female	
5	adriENNE bEll	43	Female	AI
6	EMILY JOHNSOn	36	Male	
7	edwArD EDWaRDs	21	Female	A
8	CHrisTinA MARTinez	20	Female	
9	JASmiNe aGullaR	82	Male	AI
10	ChRISTopher BerG	58	Female	A
11	michELe daniELs	72	Male	
12	aaRon MARTiNeZ	38	Female	

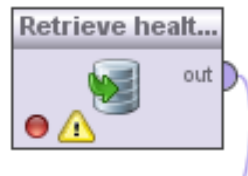
App



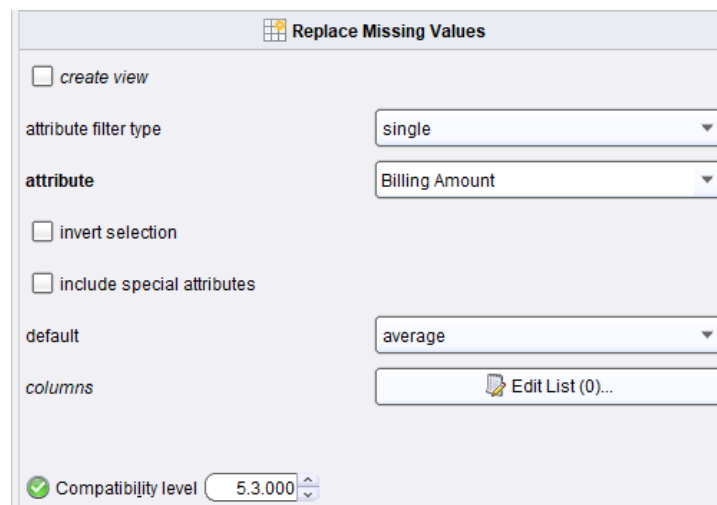
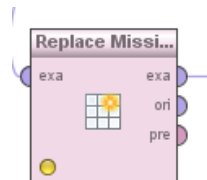
1. Load data



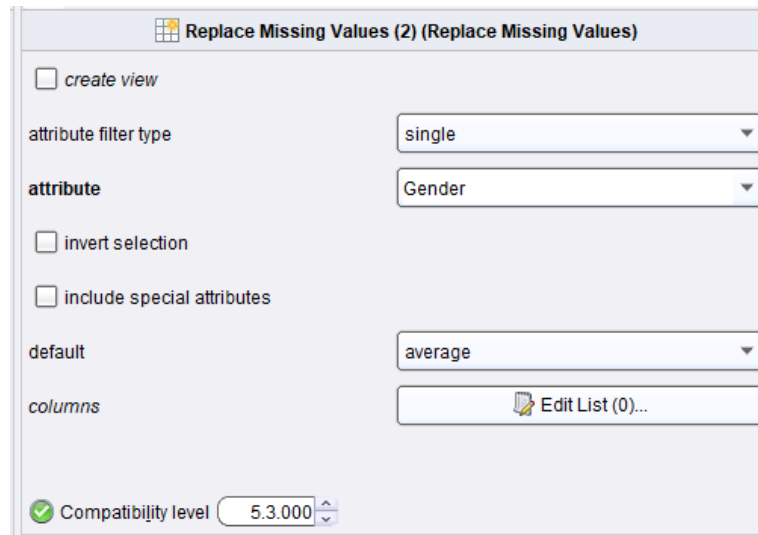
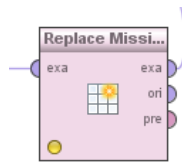
2. Drag to retrieve data



3. Drag replacing missing value port



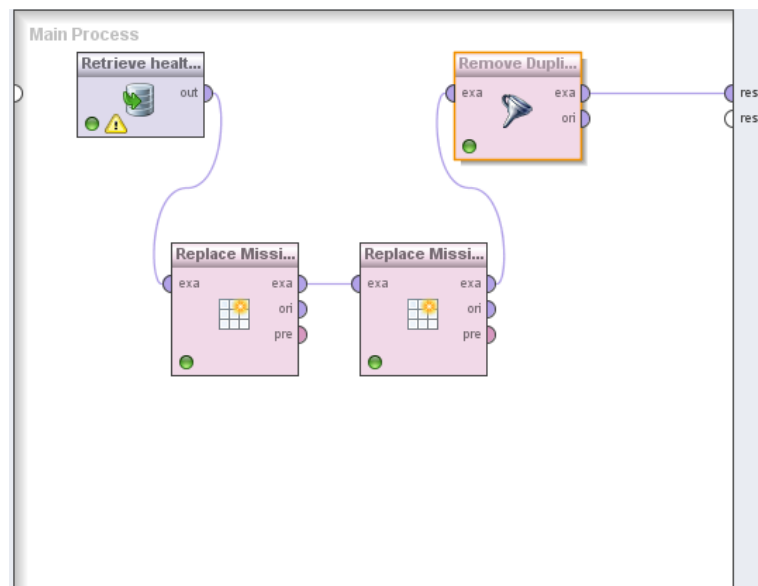
4. Drag another replacing missing value port



5. Remove duplicates



Results



ExampleSet (54966 examples, 0 special attributes, 15 regular attributes)										
View Filter (54966 / 54966):								all		
Row No.	Gender	Billing Amo...	Name	Age	Blood Type	Medical Co...	Date of Ad...	Doctor	Hospital	Insurance
1	Male	18856.281	Bobby Jacks	30	B-	Cancer	2024-01-31	Matthew Sm	Sons and Mi	Blue Cro
2	Male	33643.327	LesLie TErR	62	A+	Obesity	2019-08-20	Samantha D	Kim Inc	Medicare
3	Female	27955.096	DaNnY sMit	76	A-	Obesity	2022-09-22	Tiffany Mitch	Cook PLC	Aetna
4	Female	37909.782	andrEw waT	28	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez F	Medicare
5	Female	14238.318	adriENNE b	43	AB+	Cancer	2022-09-19	Kathleen Ha	White-White	Aetna
6	Male	48145.111	EMILY JOHN	36	A+	Asthma	2023-12-20	Taylor Newtc	Nunez-Hum	UnitedHe
7	Female	19580.872	edwArD EDV	21	AB-	Diabetes	2020-11-03	Kelly Olson	Group Midd	Medicare
8	Female	45820.463	CHrisTinA M	20	A+	Cancer	2021-12-28	Suzanne Th	Powell Robi	Cigna
9	Male	50119.223	JASmiNe aC	82	AB+	Asthma	2020-07-01	Daniel Fergu	Sons Rich a	Cigna
10	Female	19784.631	ChRISTophe	58	AB-	Cancer	2021-05-23	Heather Day	Padilla-Walk	UnitedHe
11	Male	12576.796	mlchEILe da	72	O+	Cancer	2020-04-19	John Duncal	Schaefer-Po	Medicare

LAB 4

DATA PREPROCESSING

Definition: Data preprocessing is the step of transforming raw data into a clean, structured, and usable format before applying mining or machine learning algorithms.

Purpose: Enhances accuracy, efficiency, and reliability of models.

Relation to Data Cleaning: While cleaning removes errors, preprocessing **transforms and optimizes** data for analysis.

Major Data Preprocessing Techniques

1. Data Cleaning

Handling missing values (deletion, imputation).

Removing duplicates and inconsistencies.

2. Data Transformation

Normalization: Scale values into a specific range (e.g., 0–1).

Standardization: Transform data to have mean = 0 and variance = 1.

Encoding: Convert categorical variables into numerical form (e.g., one-hot encoding).

Discretization: Convert continuous attributes into categorical bins.

3. Data Reduction

Feature Selection: Keep only relevant attributes.

Dimensionality Reduction: Techniques like PCA (Principal Component Analysis).

Aggregation: Summarize data (e.g., monthly sales totals).

Sampling: Select representative subsets of data.

4. Data Integration

Combine data from multiple sources into a unified dataset.

Resolve schema conflicts and redundancies.

Example

Dataset: Iris dataset.

Steps:

1. Normalize sepal and petal measurements to [0,1].
2. Encode species labels (Setosa, Versicolor, Virginica) using one-hot encoding.

3. Apply PCA to reduce 4 features → 2 principal components.
4. Visualize clusters in 2D space.

Result: A well-preprocessed dataset ready for classification or clustering.

Difference Between Cleaning & Preprocessing

Aspect	Data Cleaning	Data Preprocessing
Focus	Correcting errors	Transforming & optimizing
Techniques	Imputation, deduplication	Normalization, encoding, reduction
Goal	Ensure accuracy	Ensure usability & efficiency

Lab Work

Dataset

1. Summary Statistics

```
[1]: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')
```

```
[9]: print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

```
[1]: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')

[9]: print("Summary Statistics:")
display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

2. Normalization

```
[4]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

```
[4]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

3. Standardization

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

4. Discretization

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                          bins=3,
                                          labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

Discretized Data (First 5 rows):

	Income	Income_Category	Age	Age_Bins
0	81788	Medium	56	2.0
1	102879	Medium	69	2.0
2	58827	Low	46	1.0
3	127188	High	32	0.0
4	25655	Low	60	2.0

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                          bins=3,
                                          labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

Discretized Data (First 5 rows):

	Income	Income_Category	Age	Age_Bins
0	81788	Medium	56	2.0
1	102879	Medium	69	2.0
2	58827	Low	46	1.0
3	127188	High	32	0.0
4	25655	Low	60	2.0

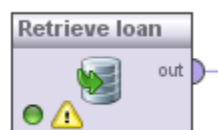
App



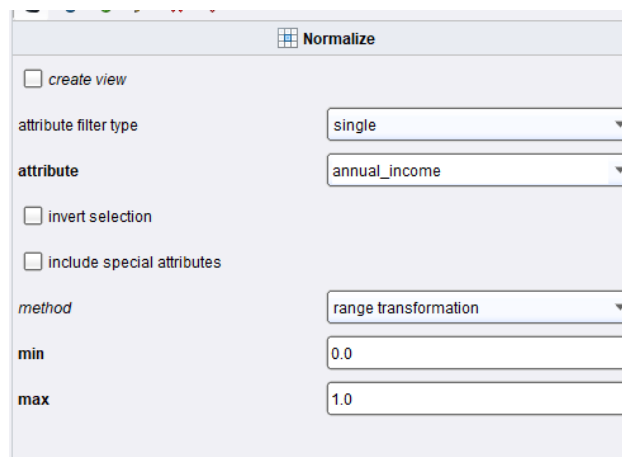
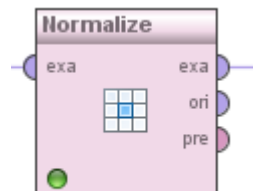
1. Load data



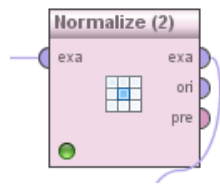
2. Drag to retrieve data



3. Drag normalize port



4. Drag another normalize port



Normalize (2) (Normalize)

☐ create view

attribute filter type: single

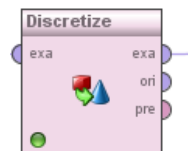
attribute: loan_amount

☐ invert selection

☐ include special attributes

method: Z-transformation

5. Drag discretize by binning port



Discretize (Discretize by Binning)

☐ create view

attribute filter type: single

attribute: monthly_income

☐ invert selection

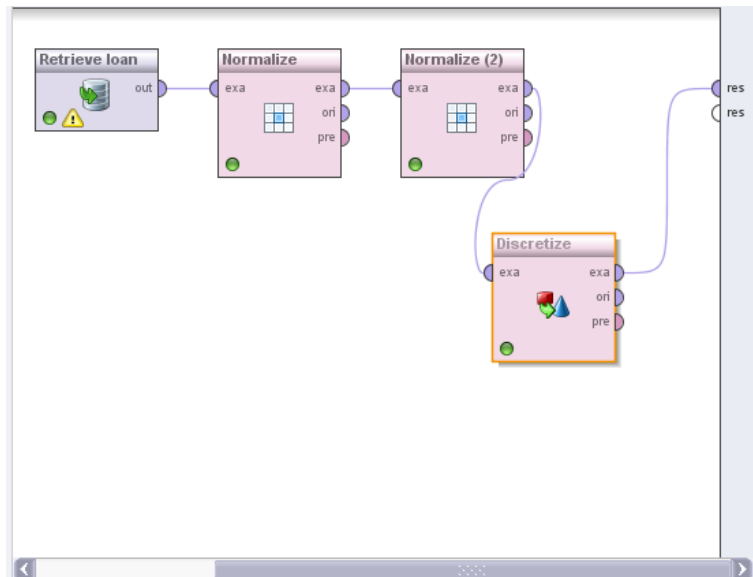
☐ include special attributes

number of bins: 3

☐ define boundaries

range name type: long

Result



ExampleSet (20000 examples, 0 special attributes, 22 regular attributes) View Filter (20000 / 20000): all

Row No.	monthly_inc...	loan_amount	annual_inc...	age	gender	marital_stat...	education_l...	employemen...	debt_to_inc...	credit_sc
1	range1 [-∞ -	0.238	0.046	59	Male	Married	Master's	Employed	0.074	743
2	range1 [-∞ -	0.876	0.036	72	Female	Married	Bachelor's	Employed	0.219	531
3	range1 [-∞ -	-1.336	0.051	49	Female	Single	High School	Employed	0.234	779
4	range1 [-∞ -	-0.022	0.015	35	Female	Single	High School	Employed	0.264	809
5	range1 [-∞ -	0.165	0.049	63	Other	Single	Other	Employed	0.260	663
6	range1 [-∞ -	-0.280	0.126	28	Male	Single	High School	Employed	0.081	774
7	range1 [-∞ -	-1.072	0.047	41	Male	Single	Bachelor's	Unemployec	0.165	742
8	range1 [-∞ -	0.001	0.118	59	Female	Single	Bachelor's	Employed	0.135	810
9	range1 [-∞ -	-1.700	0.144	39	Other	Single	High School	Employed	0.074	724
10	range1 [-∞ -	-0.048	0.120	43	Female	Single	High School	Employed	0.375	796
11	range1 [-∞ -	1.129	0.079	31	Female	Married	High School	Self-employ	0.166	722

LAB 5

MARKET BASKET ANALYSIS

Definition: A data mining technique used to discover associations between items purchased together in transactions.

Purpose: Helps businesses identify product bundling, cross-selling opportunities, and customer buying patterns.

Key Concepts in MBA

1. Association Rules

Rules of the form: $X \rightarrow Y$ (If X is purchased, Y is likely purchased).

2. Measures

- **Support:** Frequency of itemset occurrence.
- **Confidence:** Probability that Y is purchased given X.
- **Lift:** Strength of association compared to random chance.

Common Algorithms

- **Apriori Algorithm:** Iteratively generates frequent itemsets using support thresholds.
- **FP-Growth Algorithm:** Uses a compressed tree structure for faster mining.
- **Eclat Algorithm:** Uses depth-first search with intersection-based approach.

Applications of Market Basket Analysis

- **Retail:** Product placement, promotions, bundling.
- **E-commerce:** Recommendation systems (“Customers also bought...”).
- **Healthcare:** Identifying co-occurring symptoms or treatments.
- **Finance:** Detecting patterns in transaction behaviors.

Example

Dataset: Grocery transactions.

Rules Generated:

$\{\text{Milk}\} \rightarrow \{\text{Bread}\}$, Support = 0.3, Confidence = 0.7, Lift = 1.5

$\{\text{Diapers}\} \rightarrow \{\text{Beer}\}$, Support = 0.2, Confidence = 0.6, Lift = 2.0

Interpretation:

Customers buying milk often buy bread.

Diaper buyers are strongly associated with beer purchases (classic MBA example).

Lab Work

Dataset

 HEERAL DEDHIA · UPDATED 6 YEARS AGO

711

Code

Download

Groceries dataset

Dataset of 38765 rows for Market Basket Analysis

[Data Card](#) [Code \(188\)](#) [Discussion \(2\)](#) [Suggestions \(0\)](#)

About Dataset

Association Rule Mining

Market Basket Analysis is one of the key techniques used by large retailers to uncover associations between items. It works by looking for combinations of items that occur together frequently in transactions. To put it another way, it allows retailers to identify relationships between the items that people buy.

Usability ⓘ
10.00**License**
[GPL 2](#)**Expected update frequency**
Quarterly

1. Import libraries

[1]:

```
import pandas as pd
```

2. Load dataset

[3]:

```
try:
    df = pd.read_csv('Groceries_dataset.csv')
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("Error: Place 'groceries_dataset.csv' in this folder.")
    exit()
```

Dataset loaded successfully!

3. Build basket

[5]:

```
basket_series = df.groupby(['Member_number', 'Date'])['itemDescription'].apply(se
total_baskets = len(basket_series)
```

4. Track items

```
[7]:
item_counts = {}
pair_counts = {}

for basket in basket_series:
    # Track single items
    for item in basket:
        item_counts[item] = item_counts.get(item, 0) + 1

    # Track pairings
    basket_items = list(basket)
    for i in range(len(basket_items)):
        for j in range(i + 1, len(basket_items)):
            pair = tuple(sorted([basket_items[i], basket_items[j]]))
            pair_counts[pair] = pair_counts.get(pair, 0) + 1
```

5. Generate baskets

```
[9]:
print("\n\nSTRATEGIC RULES")
min_support_count = total_baskets * 0.005 # Look for combinations appearing in a

rules_found = 0
for pair, count in pair_counts.items():
    if count >= min_support_count:
        itemA, itemB = pair
        support_pair = count / total_baskets

        # Test Rule: If A -> B
        support_A = item_counts[itemA] / total_baskets
        support_B = item_counts[itemB] / total_baskets

        confidence_A_to_B = support_pair / support_A
        lift_A_to_B = confidence_A_to_B / support_B

        if lift_A_to_B > 1.0: # Only print meaningful relationships
            print(f"Rule: If a customer buys [{itemA}], they are likely to buy [{itemB}]")
            print(f"      [Support: {support_pair:.4f} | Confidence: {confidence_A_to_B:.4f} | Lift: {lift_A_to_B:.4f}]")
            rules_found += 1
            if rules_found >= 5: break # Cap at top 5 recommendations

print(f"\nTotal transactions analyzed: {total_baskets}")
print("=====")
```

```
\STRATEGIC RULES
Rule: If a customer buys [sausage], they are likely to buy [yogurt]
      [Support: 0.0057 | Confidence: 0.0952 | Lift: 1.1090]
Rule: If a customer buys [frankfurter], they are likely to buy [other vegetables]
      [Support: 0.0051 | Confidence: 0.1363 | Lift: 1.1161]
Rule: If a customer buys [sausage], they are likely to buy [soda]
      [Support: 0.0059 | Confidence: 0.0986 | Lift: 1.0150]
```

```
Total transactions analyzed: 14963
```

```
=====
<>:1: SyntaxWarning: invalid escape sequence '\S'
<>:1: SyntaxWarning: invalid escape sequence '\S'
C:\Users\hp\AppData\Local\Temp\ipykernel_2388\130383414.py:1: SyntaxWarning: invalid escape sequence '\S'
print("\n\nSTRATEGIC RULES")
```

```
[ ]:
```

LAB 6

APRIORI ALGORITHM

Definition: Apriori is a classic algorithm used in **Market Basket Analysis** to identify frequent item sets and generate association rules.

Purpose: Helps discover relationships between items in large transactional datasets.

Key Idea: Uses the principle that *all subsets of a frequent itemset must also be frequent*.

Steps of the Apriori Algorithm

1. Set minimum support and confidence thresholds.
2. Generate candidate item sets of size 1 (single items).
3. Prune candidates that do not meet minimum support.
4. Generate larger item sets (size $k+1$) from frequent item sets of size k .
5. Repeat pruning until no more frequent item sets can be generated.
6. Generate association rules from frequent item sets.
7. Evaluate rules using support, confidence, and lift.

Key Measures

- **Support:** Frequency of occurrence of an itemset.
- **Confidence:** Strength of implication (likelihood of Y given X).
- **Lift:** Measure of how much more likely X and Y occur together compared to random chance.

Advantages & Limitations

Advantages	Limitations
Simple and easy to implement	Computationally expensive for large datasets
Provides interpretable rules	Generates many redundant rules
Useful for retail and recommendation systems	Requires careful tuning of support/confidence thresholds

Example

Dataset: Grocery transactions.

Steps:

1. Minimum support = 0.3, confidence = 0.7.
2. Frequent itemset: {Milk, Bread}.

3. Rule: {Milk} → {Bread}, Support = 0.3, Confidence = 0.7, Lift = 1.5.

Interpretation: Customers buying milk are likely to buy bread, useful for promotions or product placement.

Lab Work

1. Importing libraries:

```
[2]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import apriori, association_rules
```

2. Load Dataset

```
[3]: df = pd.read_csv('transaction_data.csv')
      df.head()
```

```
[3]:
```

	Userld	Transactionld	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

```
[6]: df = df.dropna()
      # Remove negative or zero quantities (if exist)
      df = df[df['NumberOfItemsPurchased'] > 0]
      df.head()
```

```
[6]:
```

	Userld	Transactionld	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

3. Convert into Basket Format

```
[7]: basket = df.groupby(['TransactionId', 'ItemDescription'])['NumberOfItemsPurchased'] \
      .sum().unstack().reset_index().fillna(0)
      basket.head()
```

```
[7]:
```

ItemDescription	Transactionld	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OV WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	...	returned	taig adjust	test
0	5900015	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
1	5900026	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
2	5900037	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
3	5900048	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
4	5900059	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0

5 rows × 4078 columns

4. Convert Values into Binary

```
[8]: def encode(x):
      if x > 0:
          return 1
      else:
          return 0

      basket_encoded = basket.set_index('TransactionId').applymap(encode)

      basket_encoded.head()
```

C:\Users\star\AppData\Local\Temp\ipykernel_11340\1282610810.py:7: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead

```
      basket_encoded = basket.set_index('TransactionId').applymap(encode)
```

```
[8]:
```

ItemDescription	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	SPACEBOY BABY GIFT SET	...	returned	taig adjust	test
TransactionId														
5900015	0	0	0	0	0	0	0	0	0	0	...	0	0	0
5900026	0	0	0	0	0	0	0	0	0	0	...	0	0	0

5. Apply Apriori Algorithm

```
[10]: rules = association_rules(frequent_itemsets,
                                metric="confidence",
                                min_threshold=0.6)

      rules.head()
```

```
[10]: antecedents consequents antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support support
```

6. Display Important Rules

```
[11]: rules = rules.sort_values(by='confidence', ascending=False)

      rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

```
[11]: antecedents consequents support confidence lift
```

7. Filter Strong Rules

```
[12]: strong_rules = rules[(rules['confidence'] > 0.7) & (rules['lift'] > 1)]

      strong_rules
```

```
[12]: antecedents consequents antecedent consequent support confidence lift representativity leverage conviction zhangs_metric jaccard certainty kulczynski
      support support
```

App



1. Read csv file



2. Use partitioning node



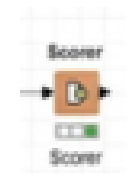
3. Drag decision tree node



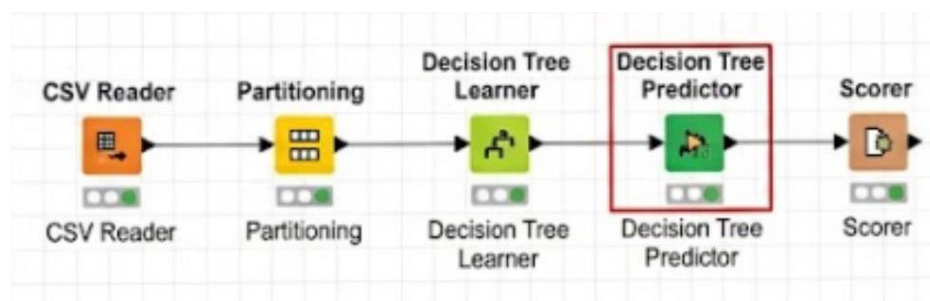
4. Also, decision tree predictor node



5. At last score



Result



LAB 7

FP-TREE & FP-GROWTH

Definition: FP-Tree is a compact data structure that represents frequent itemsets without generating candidate sets explicitly.

Purpose: Reduces the number of database scans compared to Apriori.

Key Idea: Items are stored in a prefix tree structure, allowing shared paths for common transactions.

FP-Tree Characteristics

- Built using two database scans:

First scan → Identify frequent items and sort them by frequency.

Second scan → Build the FP-Tree using ordered items.

- Each node represents an item and its frequency count.
- Paths represent transactions, with overlapping items sharing branches.

FP-Growth Algorithm

Definition: An algorithm that mines frequent itemsets directly from the FP-Tree.

Steps:

1. Build FP-Tree from dataset.
2. Extract conditional pattern bases (subsets of transactions relevant to an item).
3. Construct conditional FP-Trees recursively.
4. Generate frequent itemsets without candidate generation.

Comparison: Apriori vs FP-Growth

Aspect	Apriori	FP-Growth
Candidate Generation	Yes, explicit	No, avoids candidate generation
Database Scans	Multiple	Only two
Efficiency	Slower for large datasets	Faster and more scalable
Memory Usage	Higher	Lower (compact tree structure)

Applications

- **Retail:** Product bundling and promotions.
- **E-commerce:** Recommendation systems.
- **Healthcare:** Identifying co-occurring symptoms or treatments.
- **Finance:** Detecting transaction patterns.

Example

Dataset: Grocery transactions.

Steps:

1. Frequent items: {Milk, Bread, Butter}.
2. FP-Tree built with shared paths for transactions.
3. FP-Growth extracts itemsets: {Milk, Bread}, {Milk, Butter}, {Bread, Butter}.
4. Rules: {Milk} $\rightarrow$ {Bread}, Confidence = 0.75, Lift = 1.6.

Interpretation: FP-Growth quickly identifies strong associations without generating large candidate sets.

Lab Work

1. Import Libraries

```
[1]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import fpgrowth, association_rules
```

2. Load Dataset

```
[3]: df = pd.read_csv("retail__dataset.csv")
```

```
[7]: df.head()
```

```
[7]:
```

	TransactionID	Item
0	T1	Laptop
1	T1	Mouse
2	T1	Laptop Bag
3	T2	Headphones
4	T2	Mouse

3. Convert into Transaction Format

```
[8]: transactions = df.groupby('TransactionID')['Item'].apply(list)

# Convert to list of lists
transactions = list(transactions)

# Show sample
transactions[:5]

[8]: [['Laptop', 'Mouse', 'Laptop Bag'],
      ['Laptop', 'Mouse', 'USB Hub'],
      ['Printer', 'Paper Pack'],
      ['Monitor', 'Mouse'],
      ['Camera', 'Memory Card']]
```

4. One-Hot Encoding

```
[9]: te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

df_encoded = pd.DataFrame(te_array, columns=te.columns_)

df_encoded.head()
```

```
[9]:
```

	Bluetooth Adapter	Camera	Camera Bag	Cooling Pad	HDMI Cable	Headphones	Ink Cartridge	Keyboard	Laptop	Laptop Bag	...	Paper Pack	Phone Case	Printer	Screen Protector	Smartphone	Stylus Pen	Tablet
0	False	False	False	False	False	False	False	False	True	True	...	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	True	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
4	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False

5 rows × 23 columns

5. Apply FP-growth

```
[13]: # FP-Growth
frequent_itemsets = fpgrowth(df_encoded, min_support=0.1, use_colnames=True)

print(frequent_itemsets)
```

	support	itemsets
0	0.333333	(Mouse)
1	0.222222	(Laptop)
2	0.111111	(Printer)
3	0.166667	(Monitor)
4	0.166667	(Camera)
5	0.166667	(Keyboard)
6	0.111111	(Tablet)
7	0.111111	(Smartphone)
8	0.111111	(Headphones)
9	0.166667	(Mouse, Laptop)
10	0.111111	(Mouse, Monitor)
11	0.111111	(Keyboard, Mouse)

6. Generate Association Rules

```
[15]: # Association Rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.3)

display(rules)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
0	(Mouse)	(Laptop)	0.333333	0.222222	0.166667	0.500000	2.25	1.0	0.092593	1.555556	0.833333	0.428571	0.357143	0.62
1	(Laptop)	(Mouse)	0.222222	0.333333	0.166667	0.750000	2.25	1.0	0.092593	2.666667	0.714286	0.428571	0.625000	0.62
2	(Mouse)	(Monitor)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50
3	(Monitor)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
4	(Keyboard)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
5	(Mouse)	(Keyboard)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50

```
[16]: rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

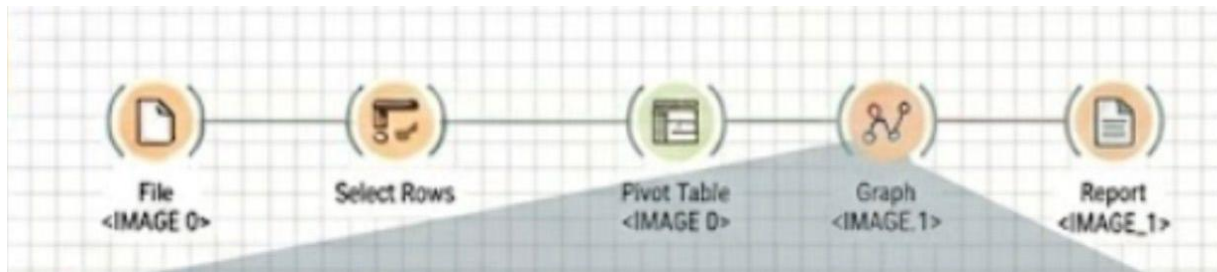
```
[16]:
```

	antecedents	consequents	support	confidence	lift
0	(Mouse)	(Laptop)	0.166667	0.500000	2.25
1	(Laptop)	(Mouse)	0.166667	0.750000	2.25
2	(Mouse)	(Monitor)	0.111111	0.333333	2.00
3	(Monitor)	(Mouse)	0.111111	0.666667	2.00
4	(Keyboard)	(Mouse)	0.111111	0.666667	2.00
5	(Mouse)	(Keyboard)	0.111111	0.333333	2.00

App



Result



LAB 8

PYTHON BASICS FOR DATA MINING

Python for Data Mining

- **Ease of Use:** Simple syntax, beginner-friendly.
- **Rich Libraries:** Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn.
- **Community Support:** Widely used in academia and industry.
- **Integration:** Works with databases, big data tools, and machine learning frameworks.

Python Basics

- **Data Types:** int, float, string, list, tuple, dictionary.
- **Control Structures:** if-else, loops (for, while).
- **Functions:** Defining reusable blocks of code.
- **Modules & Libraries:** Importing external packages (import pandas as pd).

Essential Libraries for Data Mining

Library	Purpose	Example Usage
NumPy	Numerical computing, arrays	<code>np.array([1,2,3])</code>
Pandas	Data manipulation, Data Frames	<code>pd.read_csv("data.csv")</code>
Matplotlib	Visualization	<code>plt.plot(x,y)</code>
Seaborn	Statistical plots	<code>sns.heatmap(df.corr())</code>
Scikit-learn	Machine learning	<code>from sklearn.tree import DecisionTreeClassifier</code>

Applications

- **Retail:** Customer segmentation.
- **Healthcare:** Predicting patient outcomes.
- **Finance:** Fraud detection.
- **E-commerce:** Recommendation systems.

Example

Dataset: Titanic dataset.

Steps:

1. Import Libraries

```
import pandas as pd
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier
```

2. Load dataset

```
df = pd.read_csv("titanic.csv")
```

3. Explore

```
print(df.head())
print(df.isnull().sum())
```

4. Preprocess

```
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Gender'] = df['Gender'].map({'male':0, 'female':1})
```

5. Model

```
X = df[['Age','Sex','Pclass']]
y = df['Survived']
model = DecisionTreeClassifier()
model.fit(X,y)
```

6. Display

```
print("Accuracy:", model.score(X,y))
```

Result: A simple decision tree predicts survival with basic preprocessing.

Lab Work

1. Import Library

```
[1]: import pandas as pd
```

2. Load Dataset

```
[2]: data = pd.read_csv("Loan Prediction.zip")
```

```
[3]: data.head()
```

```
[3]:
```

	Id	Income	Age	Experience	Married/Single	House_Ownership	Car_Ownership	Profession	CITY	STATE	CURRENT_JOB_YRS
0	1	1303834	23	3	single	rented	no	Mechanical_engineer	Rewa	Madhya_Pradesh	3
1	2	7574516	40	10	single	rented	no	Software_Developer	Parbhani	Maharashtra	9
2	3	3991815	66	4	married	rented	no	Technical_writer	Alappuzha	Kerala	4
3	4	6256451	41	2	single	rented	yes	Software_Developer	Bhubaneswar	Odisha	2
4	5	5768871	47	11	single	rented	no	Civil_servant	Tiruchirappalli[10]	Tamil_Nadu	3

3. Create a Class

```
[4]: class DataProcessor:

    def __init__(self, dataframe):
        self.df = dataframe

    def show_info(self):
        return self.df.info()

    def handle_missing(self):
        return self.df.fillna(self.df.mean(numeric_only=True))

    def remove_duplicates(self):
        return self.df.drop_duplicates()

    def clean_columns(self):
        self.df.columns = [col.lower() for col in self.df.columns]
        return self.df

    def summary(self):
        return self.df.describe()
```

4. Create Object of Class

```
[5]: processor = DataProcessor(data)
```

5. Apply Pre-Processing Methods

```
[6]: data = processor.clean_columns()
data = processor.handle_missing()

# Remove duplicates
data = processor.remove_duplicates()
```

6. Use Functions

```
[7]: # Show dataset info
processor.show_info()

# Show summary
processor.summary()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252000 entries, 0 to 251999
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   id                   252000 non-null  int64
1   income               252000 non-null  int64
2   age                  252000 non-null  int64
3   experience            252000 non-null  int64
4   married/single       252000 non-null  object
5   house_ownership      252000 non-null  object
6   car_ownership        252000 non-null  object
7   profession           252000 non-null  object
8   city                 252000 non-null  object
9   state                252000 non-null  object
10  current_job_yrs       252000 non-null  int64
11  current_house_yrs     252000 non-null  int64
```

```
[7]:
```

	id	income	age	experience	current_job_yrs	current_house_yrs	risk_flag
count	252000.000000	2.520000e+05	252000.000000	252000.000000	252000.000000	252000.000000	252000.000000
mean	126000.500000	4.997117e+06	49.954071	10.084437	6.333877	11.997794	0.123000
std	72746.278255	2.878311e+06	17.063855	6.002590	3.647053	1.399037	0.328438
min	1.000000	1.031000e+04	21.000000	0.000000	0.000000	10.000000	0.000000
25%	63000.750000	2.503015e+06	35.000000	5.000000	3.000000	11.000000	0.000000
50%	126000.500000	5.000694e+06	50.000000	10.000000	6.000000	12.000000	0.000000
75%	189000.250000	7.477502e+06	65.000000	15.000000	9.000000	13.000000	0.000000
max	252000.000000	9.999938e+06	79.000000	20.000000	14.000000	14.000000	1.000000

7. List and Dictionary

```
[8]: # List example: column names
columns_list = list(data.columns)
print(columns_list)

# Dictionary example: data types
dtype_dict = dict(data.dtypes)
print(dtype_dict)

['id', 'income', 'age', 'experience', 'married/single', 'house_ownership', 'car_ownership', 'profession', 'city', 'state', 'current_job_yrs', 'current_h
ouse_yrs', 'risk_flag']
{'id': dtype('int64'), 'income': dtype('int64'), 'age': dtype('int64'), 'experience': dtype('int64'), 'married/single': dtype('O'), 'house_ownership': d
type('O'), 'car_ownership': dtype('O'), 'profession': dtype('O'), 'city': dtype('O'), 'state': dtype('O'), 'current_job_yrs': dtype('int64'), 'current_h
ouse_yrs': dtype('int64'), 'risk_flag': dtype('int64')}
```


LAB 9

DECISION TREE

1. Introduction & Theoretical Background

1.1 Definition

A **Decision Tree** is a non-parametric, supervised machine learning algorithm used for both classification and regression tasks. It is structured as a hierarchical flowchart where:

- **Root Node:** Represents the topmost choice or entire population, which gets divided into two or more homogeneous sets.
- **Internal/Decision Nodes:** Represent test conditions or attributes (e.g., *Is the customer's account balance..*).
- **Branches/Edges:** Represent the outcomes of a test node (e.g., *Yes* or *No*).
- **Leaf Nodes:** Represent the final class labels or predicted target distributions (e.g., *Will Purchase*, *Will Not Purchase*).

1.2 Splitting Criteria

To build an optimal tree, the algorithm must choose which column splits the data most cleanly at each node. This is calculated using mathematical metrics of purity:

- **Entropy:** Measures the degree of impurity, randomness, or disorder in a dataset segment. A completely uniform mix of classes has an entropy of 1, while a perfectly pure split has an entropy of 0.
- **Information Gain:** The measure of net reduction in entropy after a dataset is split on a specific attribute. The algorithm chooses the attribute that yields the highest Information Gain.
- **Gini Index (Gini Impurity):** Measures the probability of an incorrect classification of a randomly chosen element if it were randomly labeled according to the class distribution in the subset. It ranges between 0 (fully pure) and 0.5 (equally distributed binary split).

2. Real-World Problem Statement

Problem: A corporate bank or e-commerce platform wants to optimize its targeted cross-selling marketing strategy. Blindly reaching out to every client wastes capital and creates customer fatigue.

Objective: Induce a Decision Tree model to predict customer purchase behavior (Target Variable: deposit or purchase right arrow *Yes / No*) based on user demographics and historical account activities, enabling high-conversion micro-targeting.

3. Practical Objectives

1. **Data Partitioning:** Segment a raw historical marketing dataset into a **Training Set (70%)** to visually grow the tree branches, and a **Testing Set (30%)** to strictly validate performance on unseen data.
2. **Tree Induction:** Apply information-theoretic criteria (Gini Index or Entropy) to identify dominant splitting nodes.
3. **Pruning:** Configure tree-depth parameters (e.g., maximum depth or minimal leaf size) to implement Minimal Description Length (MDL) pruning, which halts growth to avoid **overfitting** (where a model memorizes training noise instead of learning structural trends).

4. Mathematical Evaluation Metrics & KPIs

To measure the predictive capabilities of the model against the validation slice, a **Confusion Matrix** is computed to track True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). From this matrix, four crucial KPIs must be recorded:

4.1 Accuracy

The overall proportion of correct classifications out of all tested customer profiles.

4.2 Precision

Out of all customers the model *predicted* would buy a product, what percentage actually bought it? High precision minimizes wasted marketing budget on false leads.

4.3 Recall (Sensitivity)

Out of all the customers who *actually* wanted to buy a product, what percentage did the model successfully identify? High recall ensures no revenue-generating opportunities are missed.

4.4 F1-Score

The harmonic mean of Precision and Recall. It provides a balanced indicator of model performance, especially when there is a significant class imbalance (e.g., if only 10% of customers actually buy).

LAB WORK

Dataset

CUST_ID	BALANCE	PURCHASE	ONEOFF	INSTALLM	CASH_AD	PURCHASE	ONEOFF	PURCHASE	CASH_AD	CASH_AD	PURCHASE	CREDIT_LI	PAYMENT_M
C10001	40.90075	0.818182	95.4	0	95.4	0	0.166667	0	0.083333	0	2	1000	201.8021
C10002	3202.467	0.909091	0	0	6442.945	0	0	0	0.25	4	0	7000	4103.033
C10003	2495.149	1	773.17	773.17	0	0	1	1	0	0	12	7500	622.0667
C10004	1666.671	0.636364	1499	1499	0	205.788	0.083333	0.083333	0	0.083333	1	1	7500
C10005	817.7143	1	16	16	0	0	0.083333	0.083333	0	0	0	1	1200
C10006	1809.829	1	1333.28	0	1333.28	0	0.666667	0	0.583333	0	0	8	1800
C10007	627.2608	1	7091.01	6402.63	688.38	0	1	1	1	0	0	64	13500
C10008	1823.653	1	436.2	0	436.2	0	1	0	1	0	0	12	2300
C10009	1014.926	1	861.49	661.49	200	0	0.333333	0.083333	0.25	0	0	5	7000
C10010	152.776	0.545455	1281.6	1281.6	0	0	0.166667	0.166667	0	0	0	3	11000

```
1. Import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# 1. Load the Marketing Dataset
# Replace 'marketing_data.csv' with your local dataset path
df = pd.read_csv('marketing_data.csv')

# --- FIX: Generate synthetic data to resolve FileNotFoundError ---
np.random.seed(42)
n_samples = 300
data = {
    'Age': np.random.randint(18, 65, n_samples),
    'EstimatedSalary': np.random.randint(20000, 150000, n_samples),
    'Purchased': np.random.choice([0, 1], n_samples, p=[0.6, 0.4]) # Simulate some purchases
}
df = pd.DataFrame(data)
print("Generated synthetic marketing data:")
print(df.head())
print("\n")

# Assuming columns: 'Age', 'EstimatedSalary' as features and 'Purchased' as target
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

# 2. Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Initialize and Train Decision Tree Classifier
clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)

# 4. Predict on Test Set
y_pred = clf.predict(X_test)

# 5. Evaluate Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("==== Evaluation Metrics ====")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")
print("Classification Report:\n", classification_report(y_test, y_pred))

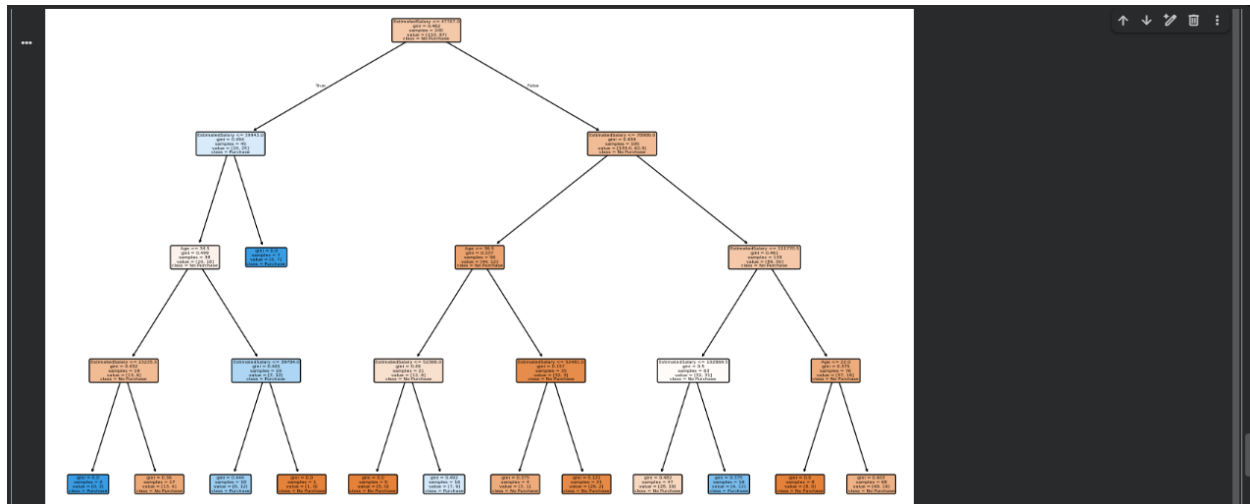
# 6. Visualize the Decision Tree
plt.figure(figsize=(15, 10))
plot_tree(clf, feature_names=X.columns, class_names=['No Purchase', 'Purchase'], filled=True, rounded=True)
plt.title('Decision Tree Structure for Marketing Insights')
plt.show()
```

Output:

```
Generated synthetic marketing data:
Age EstimatedSalary Purchased
0 55 58467 1
1 55 43325 1
2 32 139181 0
3 60 111412 0
4 25 166631 0

=== Evaluation Metrics ===
Accuracy : 0.6232
Precision: 0.5600
Recall : 0.6364
F1-Score : 0.5957

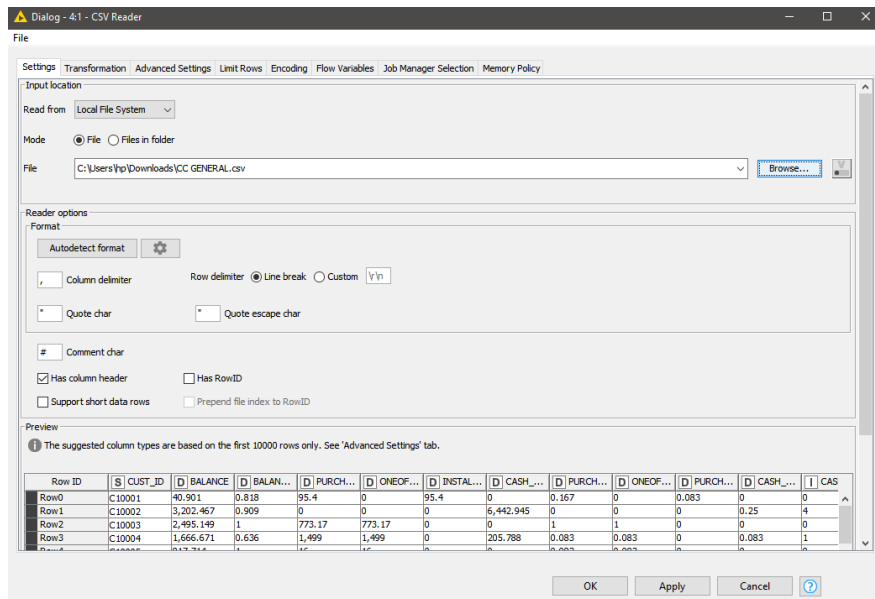
Classification Report:
      precision    recall  f1-score   support
0       0.77       0.71       0.74         38
1       0.56       0.64       0.60         22
 accuracy      macro avg       0.67       0.67       0.67         60
 weighted avg       0.69       0.68       0.69         60
```



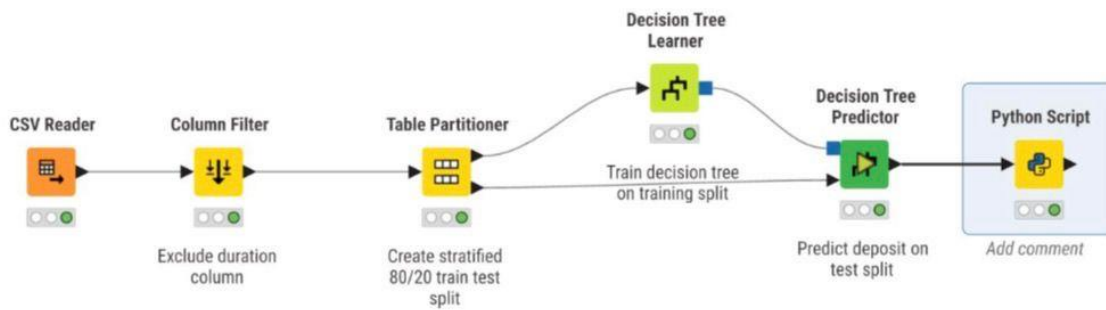
App



Load dataset



Result



LAB 10

NAÏVE BAYES AND KNN

Introduction & Theoretical Background

Definition

Naive Bayes is a supervised learning classification algorithm based on **Bayes' Theorem** with a strong (naive) assumption of conditional independence among every pair of features given the value of the class variable. Despite this oversimplification, Naive Bayes performs remarkably well on high-dimensional text datasets.

Real-World Problem Statement

- **Problem:** An organization needs to prevent malicious spam emails or phishing text messages from reaching their network infrastructure.
- **Objective:** Use Naive Bayes to classify incoming communications as **Spam** or **Ham (Legitimate)** based on word presence distributions.

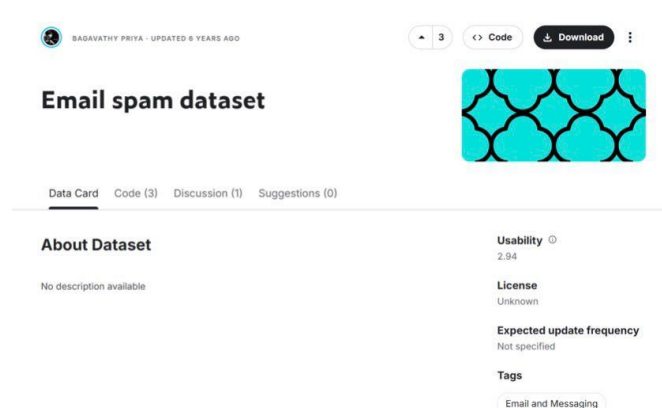
3. Target Dataset

Your manual recommends a **Social Media / Email Dataset**.

- **Kaggle Link:** [SMS Spam Collection Dataset](#)
- **Structure:** Contains a text label column (v1 as spam/ham) and a raw text message string column (v2).
- **Tokenization:** Select *Regex* (words).
- **Filtering:** Enable *Stopwords* (removes generic words like "the", "and", "is").

LAB WORK

Dataset



```
[ ] import pandas as pd

# Load dataset
df = pd.read_csv('spam.csv.csv', encoding='latin-1')

print("Dataset Shape:", df.shape)
print("\nColumns:")
print(df.columns)

print("\nFirst 5 Rows:")
print(df.head())
```

Dataset Shape: (5573, 1)

Columns:
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Ci

First 5 Rows:
ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Ci
0 ham\tok lar... Joking wif u oni...
1 spam\tFree entry in 2 a wkly comp to win FA Cu...
2 ham\tU dun say so early hor... U c already the...
3 ham\tNah I don't think he goes to usf, he live...
4 spam\tFreeMsg Hey there darling it's been 3 we...

```
[ ] # Remove extra columns if they exist
extra_cols = ['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']

for col in extra_cols:
    if col in df.columns:
        df.drop(col, axis=1, inplace=True)

# Rename columns if using common Kaggle spam dataset
if 'v1' in df.columns and 'v2' in df.columns:
    df.rename(columns={'v1': 'label', 'v2': 'message'}, inplace=True)

print(df.head())
print(df.columns)
```

ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Ci
0 ham\tok lar... Joking wif u oni...
1 spam\tFree entry in 2 a wkly comp to win FA Cu...
2 ham\tU dun say so early hor... U c already the...
3 ham\tNah I don't think he goes to usf, he live...
4 spam\tFreeMsg Hey there darling it's been 3 we...
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet...

```
[ ] print(df.columns)
```

Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet...

```
[ ] import pandas as pd

# Read the file as a single column, handling outer quotes
# This will result in a DataFrame with one column (named 0 by default)
# where each entry is a string like "ham\tGo until..."
df_raw = pd.read_csv(
    'spam.csv.csv',
    header=None,
    quotechar='"' # Handle the outer double quotes
)

# Split the single column into 'label' and 'message'
# .str.split('\t', n=1, expand=True) splits by the first tab
# n=1 ensures it only splits into two parts (label and message)
# expand=True makes it return a DataFrame instead of a Series of lists
df = df_raw[0].str.split('\t', n=1, expand=True)
df.columns = ['label', 'message'] # Assign new column names

print(df.head())
print(df.columns)
```

label message
0 ham Go until jurong point, crazy.. Available only ...
1 ham Ok lar... Joking wif u oni...
2 spam Free entry in 2 a wkly comp to win FA Cup fina...
3 ham U dun say so early hor... U c already then say...
4 ham Nah I don't think he goes to usf, he lives aro...
Index(['label', 'message'], dtype='object')

```
[1] from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Convert labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Features and target
X = df['message']
y = df['label']

# Convert text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

```
# Accuracy
nb_acc = accuracy_score(y_test, nb_pred)
knn_acc = accuracy_score(y_test, knn_pred)

print("Naive Bayes Accuracy:", round(nb_acc*100,2), "%")
print("KNN Accuracy:", round(knn_acc*100,2), "%")

# Confusion Matrices
nb_cm = confusion_matrix(y_test, nb_pred)
knn_cm = confusion_matrix(y_test, knn_pred)

print("\nNaive Bayes Confusion Matrix")
print(nb_cm)

print("\nKNN Confusion Matrix")
print(knn_cm)
```

```
*** Naive Bayes Accuracy: 98.48 %
KNN Accuracy: 90.85 %

Naive Bayes Confusion Matrix
[[944  10]
 [ 7 154]]

KNN Confusion Matrix
[[954   0]
 [102  59]]
```

```
[1] import pandas as pd

def false_positive_rate(cm):
    TN = cm[0][0]
    FP = cm[0][1]
    return FP / (FP + TN)

nb_fpr = false_positive_rate(nb_cm)
knn_fpr = false_positive_rate(knn_cm)

results = pd.DataFrame({
    'Algorithm': ['Naive Bayes', 'KNN'],
    'Accuracy (%)': [
        round(nb_acc*100,2),
        round(knn_acc*100,2)
    ],
    'False Positive Rate': [
        round(nb_fpr,4),
        round(knn_fpr,4)
    ]
})

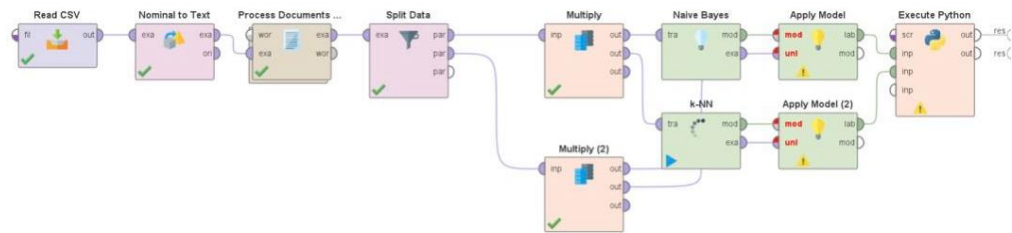
print(results)
```

```
***   Algorithm  Accuracy (%)  False Positive Rate
0  Naive Bayes      98.48      0.0105
1         KNN      90.85      0.0000
```


App



Result



LAB 11

SVM

1. Introduction & Theoretical Background

1.1 Definition

A **Support Vector Machine (SVM)** is a non-probabilistic, distance-based supervised machine learning algorithm designed to compute an optimal **Maximum Margin Hyperplane** that divides distinct classes with the absolute widest empty space possible.

1.2 Core Mathematical Concepts

- **Hyperplane:** The mathematical decision boundary that separates data coordinates. For linear binary separation.
- **Support Vectors:** The critical boundary data coordinates resting closest to the decision hyperplane. These are the vector coordinates that sustain the classification margin; if these specific coordinates shift, the entire model changes orientation.
- **The Margin:** The maximum perpendicular distance separating the parallel margins of opposing data groups. SVM maximizes this spatial width to reduce validation errors.
- **The Kernel Trick:** When points are clustered in a way that cannot be cleanly sliced by a straight linear line, SVM utilizes a mathematical **Kernel Function** to compute high-dimensional dot products, mapping complex curves into flat hyperplanes in a higher-dimensional space.

2. Real-World Problem Statement

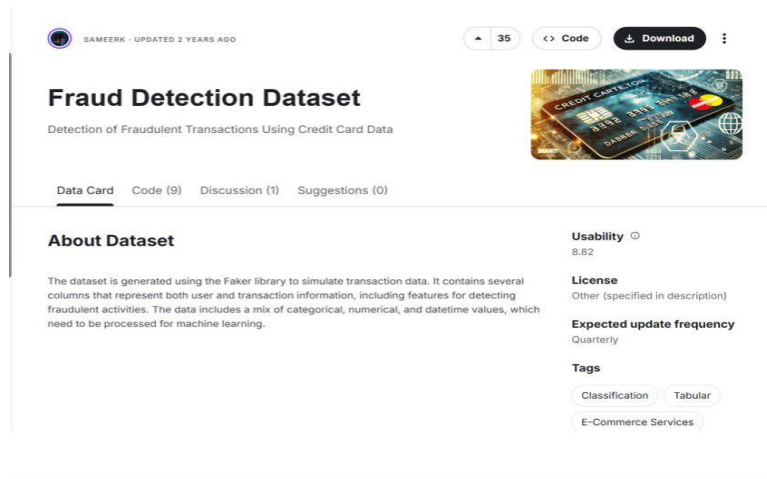
- **Problem:** A financial bank needs to assess customer transaction histories to categorize user risk profiles or flag instances of credit card fraud/defaults.
- **Objective:** Train an SVM classifier to separate high-risk defaulters from low-risk customers using financial parameters, ensuring highly stable margins.

3. Practical Objectives & Evaluation Metrics

- **Data Standardization:** Inputs must be normalized into uniform scales via z -scores to prevent high-value attributes (like total balance or credit limits) from overwhelming distance calculations.
- **Target KPIs:**
 - **Confusion Matrix:** A 2×2 layout grid computing explicitly: True Negatives (TN), False Positives (FP), False Negatives (FN), and True Positives (TP).
 - **F1-Score:** The harmonic mean balancing precision and recall limits to evaluate structural resilience against highly skewed data groups (such as fraud vs. regular credit card operations).
 - **Boundary Line Margin:** Evaluated via tracking the total number of anchor support vectors necessary to hold the optimal decision space.

LAB WORK

Dataset



Fraud Detection Dataset
Detection of Fraudulent Transactions Using Credit Card Data

About Dataset

The dataset is generated using the Faker library to simulate transaction data. It contains several columns that represent both user and transaction information, including features for detecting fraudulent activities. The data includes a mix of categorical, numerical, and datetime values, which need to be processed for machine learning.

Usability
8.82

License
Other (specified in description)

Expected update frequency
Quarterly

Tags
Classification, Tabular, E-Commerce Services

1. Import libraries

```
[ ] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, recall_
```

2. Simulate a highly imbalanced fraud detection dataset

```
[ ] np.random.seed(42)
n_samples = 2000
normal_tx = np.random.normal(loc=50, scale=10, size=(1950, 3))
fraud_tx = np.random.normal(loc=200, scale=50, size=(50, 3))
X = np.vstack([normal_tx, fraud_tx])
y = np.array([0] * 1950 + [1] * 50)
columns = ['Transaction_Amount', 'Frequency_Per_Day', 'Location_Risk_Score']
df = pd.DataFrame(X, columns=columns)
df['class'] = y
```

3. Separate features and target variables

```
[ ] X_features = df[columns]
y_target = df['class']
```

4. Train test split

```
[ ] X_train, X_test, y_train, y_test = train_test_split(
    X_features, y_target, test_size=0.25, random_state=42, stratify=y_target
)
```

5. Feature scaling

```
[1] scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

6. Initialize and train SVM with a linear kernel

```
[1] print("Training Support Vector Machine with Linear Kernel...")
svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with Linear Kernel...

SVC

SVC(kernel='linear', probability=True, random_state=42)

7. Initialize and train SVM with a radial basis function kernel

```
[1] print("Training Support Vector Machine with RBF Kernel...")
svm_rbf = SVC(kernel='rbf', gamma='scale', C=1.0, probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with RBF Kernel...

SVC

SVC(probability=True, random_state=42)

8. Generate evaluation predictions and probability estimates

```
[1] lin_preds = svm_linear.predict(X_test_scaled)
lin_probs = svm_linear.predict_proba(X_test_scaled)[:, 1]

rbf_preds = svm_rbf.predict(X_test_scaled)
rbf_probs = svm_rbf.predict_proba(X_test_scaled)[:, 1]
```

9. Display runtime KPI performance report

```
[1] print(" LAB 11: SVM KERNEL COMPARISON MATRIX ")

print(f" -> Fraud Recall Rate : {recall_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> Precision Score : {precision_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score : {roc_auc_score(y_test, lin_probs):.4f}")
print(f" -> Confusion Matrix : \n{confusion_matrix(y_test, lin_preds)}")
print("\n")
print(f"[RBF KERNEL]")
print(f" -> Fraud Recall Rate : {recall_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> Precision Score : {precision_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score : {roc_auc_score(y_test, rbf_probs):.4f}")
print(f" -> Confusion Matrix : \n{confusion_matrix(y_test, rbf_preds)}")
```

LAB 11: SVM KERNEL COMPARISON MATRIX

-> Fraud Recall Rate : 100.00%

-> Precision Score : 100.00%

-> ROC-AUC Score : 1.0000

-> Confusion Matrix :

```
[[488  0]
 [  0 12]]
```

[RBF KERNEL]

-> Fraud Recall Rate : 100.00%

-> Precision Score : 100.00%

-> ROC-AUC Score : 1.0000

-> Confusion Matrix :

```
[[488  0]
 [  0 12]]
```

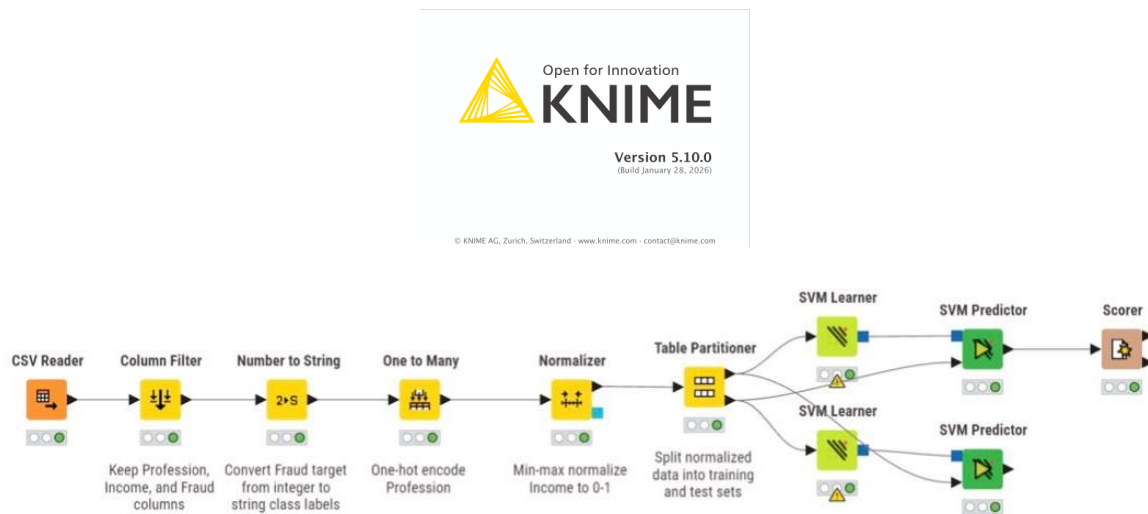
```
[ ] ▶ print("\nDetailed RBF Classification Context Matrix:")
      print(classification_report(y_test, rbf_preds, target_names=['Legitimate', 'Fraud']))
```

▼ ...

Detailed RBF Classification Context Matrix:

	precision	recall	f1-score	support
Legitimate	1.00	1.00	1.00	488
Fraud	1.00	1.00	1.00	12
accuracy			1.00	500
macro avg	1.00	1.00	1.00	500
weighted avg	1.00	1.00	1.00	500

App



LAB 12

CLUSTERING

1. Introduction & Theoretical Background

1.1 Definition

K-Means Clustering is one of the most popular unsupervised machine learning algorithms. Unlike supervised learning, it operates on unlabeled datasets to partition data points into k distinct, non-overlapping groups (clusters) based on feature similarities. It is a distance-based, centroid-driven algorithm where every data point is assigned to its nearest cluster center.

1.2 Mathematical Formulation & Algorithmic Steps

The algorithm aims to minimize the geometric distance between data points and their respective cluster centers. It minimizes the **Within-Cluster Sum of Squares (WCSS)**, also known as cluster inertia.

The iterative optimization process follows these steps:

1. **Initialization:** Choose the number of clusters k and randomly pick k data points as initial centroids.
2. **Assignment Step:** Assign each data point to the nearest centroid using the **Euclidean Distance**.
3. **Update Step:** recalculate the position of each centroid by taking the mathematical mean of all data points assigned to that cluster.
4. **Convergence:** Repeat steps 2 and 3 until the centroids stop changing position or the maximum number of iterations is reached.

2. Real-World Problem Statement

- **Problem:** A retail mall or e-commerce company collects baseline customer profile statistics but lacks clear target demographics. Bombarding the entire database with a single generic ad campaign yields poor conversion rates.
- **Objective:** Apply K-Means clustering to partition customers into distinct spending profiles based on annual earnings and behavioral scores. This allows the marketing team to run personalized, high-return micro-campaigns.

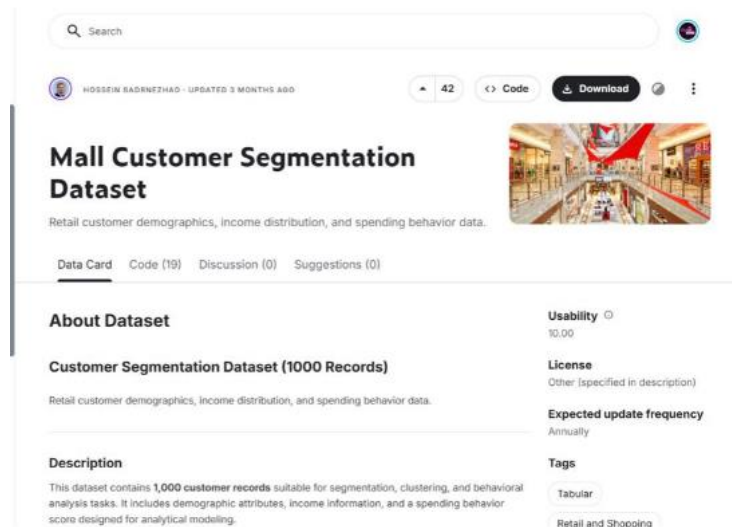
3. Target Dataset

Your manual explicitly designates the **Mall Customer Segmentation Dataset**.

- **Structure:** It contains demographic and behavioral variables: CustomerID, Gender, Age, Annual Income (k\$), and Spending Score (1-100).

LAB WORK

Dataset



1. Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.vq import kmeans2, whiten
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.metrics import silhouette_score
```

2. Load dataset

```
df = pd.read_csv('Mall_Customers.csv')
```

3. Extract feature matrix

```
# Index 2 = Age, Index 3 = Annual Income, Index 4 = Spending Score
X_features = df.iloc[:, [2, 3, 4]].values
```

4. Feature standardization

```
X_scaled = whiten(X_features)
```

5. k-means clustering

```
optimal_k = 5
centroids, kmeans_labels = kmeans2(X_scaled, optimal_k, minit='points', seed=42)
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
```

6. hierarchical agglomerative clustering

```
Z = linkage(X_scaled, method='ward', metric='euclidean')
hier_labels = fcluster(Z, optimal_k, criterion='maxclust') - 1 # Adjusted to 0-indexed bases
hier_silhouette = silhouette_score(X_scaled, hier_labels)

df['KMeans_Cluster'] = kmeans_labels
df['Hierarchical_Cluster'] = hier_labels
```

7. display KPI evaluation

```
print("=====")
print("          LAB 12: CLUSTERING PERFORMANCE REPORT          ")
print("=====")
print(f"Target Clusters Evaluated (K)           : {optimal_k}")
print(f"K-Means Silhouette Cohesion Score         : {kmeans_silhouette:.4f}")
print(f"Hierarchical Agglomerative Silhouette Score : {hier_silhouette:.4f}")
print("=====")
print("\nSample Output Consolidated with Clusters:")
print(df.iloc[:, [0, 1, 2, -2, -1]].head())
```

```
=====
          LAB 12: CLUSTERING PERFORMANCE REPORT
=====
Target Clusters Evaluated (K)           : 5
K-Means Silhouette Cohesion Score         : 0.4166
Hierarchical Agglomerative Silhouette Score : 0.3900
=====

Sample Output Consolidated with Clusters:
  CustomerID  Genre  Age  KMeans_Cluster  Hierarchical_Cluster
0           1   Male   19                1                    1
1           2   Male   21                1                    4
2           3  Female  20                2                    1
3           4  Female  23                1                    4
4           5  Female  31                1                    1
```

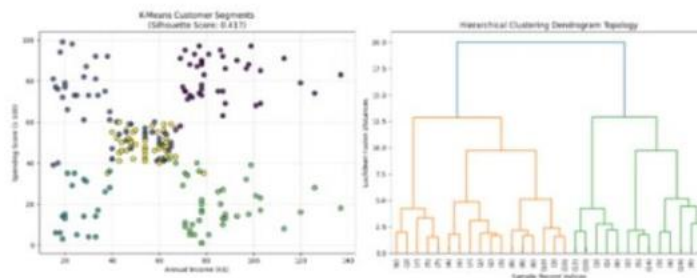
8. generate visual evaluation plots

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: K-Means Spatial Clusters Scattered Map
ax1.scatter(X_features[:, 1], X_features[:, 2],
            c=kmeans_labels, cmap='viridis', edgecolor='k', s=50, alpha=0.8)
ax1.set_title(f"K-Means Customer Segments\n(Silhouette Score: {kmeans_silhouette:.3f})")
ax1.set_xlabel(df.columns[3])
ax1.set_ylabel(df.columns[4])
ax1.grid(True, linestyle='--', alpha=0.5)

# Plot 2: Hierarchical Clustering Dendrogram Topology Tree Map
ax2.set_title("Hierarchical Clustering Dendrogram Topology")
ax2.set_xlabel("Sample Record Indices")
ax2.set_ylabel("Euclidean Fusion Distances")
dendrogram(Z, truncate_mode='lastp', p=30, ax=ax2, leaf_rotation=90, leaf_font_size=10, show_c)

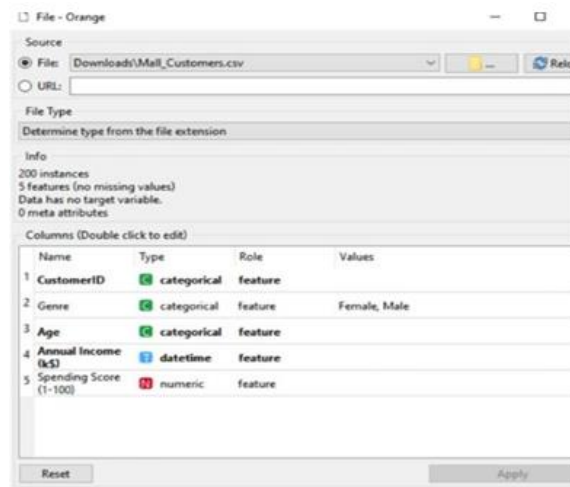
plt.tight_layout()
plt.show()
```



App



Load dataset



Result



LAB 13

OUTLIER DETECTION

1. Introduction & Theoretical Background

1.1 Definition

Hierarchical Clustering is an unsupervised machine learning algorithm used to group unlabeled data points into a tree-like structure of clusters. Unlike K-Means, it does not require you to pre-define the number of clusters (\$k\$) before running the model. Instead, it establishes an iterative hierarchy of groupings.

There are two primary strategies for building the hierarchy:

1. **Agglomerative (Bottom-Up):** Starts with every single data point treating itself as an individual cluster. At each successive iteration, the algorithm merges the closest pair of clusters together until all points are united into a single overarching cluster.
2. **Divisive (Top-Down):** Starts with all data points in one single massive cluster, and iteratively splits the most dissimilar elements into smaller clusters until every point forms its own leaf node.

1.2 Distance and Linkage Criteria

To determine which clusters should be merged (or split), the algorithm calculates mathematical similarity using a distance metric (such as **Euclidean Distance**) combined with a specific **Linkage Criterion**:

- **Single Linkage:** Measures the minimum distance between the closest pair of points in two clusters. It tends to create long, trailing, chain-like clusters.
- **Complete Linkage:** Measures the maximum distance between the furthest pair of points in two clusters. It forces compact, tightly-bound spherical clusters.
- **Average Linkage:** Compares the average distance between all pairs of points in the two clusters.
- **Ward's Method:** Minimizes the total within-cluster variance. At each step, it merges the two clusters that result in the smallest possible increase in the combined sum of squared errors (SSE).

2. Real-World Problem Statement

- **Problem:** A banking corporation or financial institution needs to secure its transaction pipelines by flagging unusual or suspicious credit card user behavior. Standard transaction rules can easily be bypassed, making automated anomaly detection vital.
- **Objective:** Apply Agglomerative Hierarchical Clustering on customer profile characteristics (such as your uploaded `CC_GENERAL.csv` data) to map structural customer segments, allowing analysts to isolate tiny, highly irregular outlier branches that deviate from mainstream banking profiles.

LAB WORK

1. Import libraries

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
```

2. Load dataset

```
df = pd.read_csv("test.csv")
print(df.columns.tolist())

FEATURE_COL_1 = 'age'
FEATURE_COL_2 = 'balance'

GROUND_TRUTH_COL = 'y'
```

3. Data loading and preprocessing

```
if not os.path.exists(FILE_PATH):
    raise FileNotFoundError(f"Could not find the dataset file at: {FILE_PATH}.")

# CRITICAL FIX: Added sep=';' to correctly split your columns
df = pd.read_csv(FILE_PATH, sep=';')

# Clean up quotes from string columns if they exist natively in the data entries
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].astype(str).str.replace('"', '').str.replace("'", '')

print(f"Dataset loaded successfully! Total Rows: {df.shape[0]}, Total Columns: {df.shape[1]}")
print("Parsed Columns:", df.columns.tolist())

# Drop missing values safely
df = df.dropna(subset=[FEATURE_COL_1, FEATURE_COL_2, GROUND_TRUTH_COL]).reset_index(drop=True)

# Convert Ground Truth 'yes'/'no' into 1 and 0 so the KPI functions can read it
df['Encoded_Ground_Truth'] = df[GROUND_TRUTH_COL].map({'yes': 1, 'no': 0})

# Extract features and scale them
X = df[[FEATURE_COL_1, FEATURE_COL_2]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

Dataset loaded successfully! Total Rows: 4521, Total Columns: 17
Parsed Columns: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'y']
```

4. Isolation forest

```
iso_forest = IsolationForest(contamination=0.05, random_state=42)
df['IF_Raw_Pred'] = iso_forest.fit_predict(X_scaled)
df['IF_Anomaly'] = df['IF_Raw_Pred'].map([1: 0, -1: 1])
```

5. Distance based

```
k_neighbors = 5
knn = NearestNeighbors(n_neighbors=k_neighbors)
knn.fit(X_scaled)
distances, _ = knn.kneighbors(X_scaled)

df['KNN_Outlier_Score'] = distances.mean(axis=1)
contamination_threshold = np.percentile(df['KNN_Outlier_Score'], 95)
df['KNN_Anomaly'] = (df['KNN_Outlier_Score'] >= contamination_threshold).astype(int)
```

6. KPI and evaluation metrics

```
def calculate_lab_kpis(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    anomaly_detection_rate = (tp / (tp + fn)) * 100 if (tp + fn) > 0 else 0
    false_alarm_rate = (fp / (fp + tn)) * 100 if (fp + tn) > 0 else 0
    return anomaly_detection_rate, false_alarm_rate

# Compute metrics using our encoded 1/0 column
if_adr, if_far = calculate_lab_kpis(df['Encoded_Ground_Truth'], df['IF_Anomaly'])
knn_adr, knn_far = calculate_lab_kpis(df['Encoded_Ground_Truth'], df['KNN_Anomaly'])

kpi_table = pd.DataFrame({
    "Evaluation Metric (KPI)": ["Anomaly Detection Rate (ADR)", "False Alarm Rate (FAR)"],
    "Isolation Forest": [f"{if_adr:.2f}%", f"{if_far:.2f}%"],
    "Distance-Based (KNN)": [f"{knn_adr:.2f}%", f"{knn_far:.2f}%"]
})

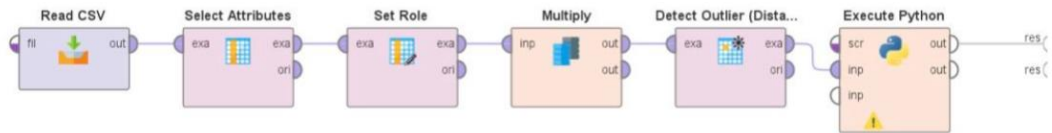
print(" EVALUATION METRICS")
print(kpi_table.to_string(index=False))
```

EVALUATION METRICS			
Evaluation Metric (KPI)	Isolation Forest	Distance-Based (KNN)	
Anomaly Detection Rate (ADR)	8.06%	7.49%	
False Alarm Rate (FAR)	4.60%	4.70%	

App



Result



LAB 14

SCRAPING AND SENTIMENT ANALYSIS

1. Introduction & Theoretical Background

1.1 Definition

Sentiment Analysis (or opinion mining) is a Natural Language Processing (NLP) subfield that applies computational text processing to systematically identify, extract, and quantify subjective emotional states and opinions hidden within unstructured text strings.

Before unstructured text can be analyzed by machine learning algorithms, it must undergo an intensive **NLP Text Preprocessing Pipeline**:

1. **Tokenization**: Chopping a continuous stream of paragraph text into individual words or structural pieces called tokens.
2. **Normalization/Lowercasing**: Converting all tokens to lowercase so that words like "Excellent" and "excellent" are mapped to the exact same numeric variable.
3. **Stop-word Removal**: Sifting out and discarding highly frequent but sentiment-neutral connector words (such as "the", "is", "at", "which") that contribute structural noise without emotional value.
4. **Lemmatization / Stemming**: Reducing words to their base linguistic dictionary root form (e.g., "loving", "loves", and "loved" all reduce down to "love").

1.2 Mathematical Formulation & Sentiment Metrics

Sentiment mapping can be accomplished via machine learning classifiers or **Lexicon-Based Rules** (using pre-built dictionaries like VADER or TextBlob). Lexicons evaluate text by calculating two key metrics:

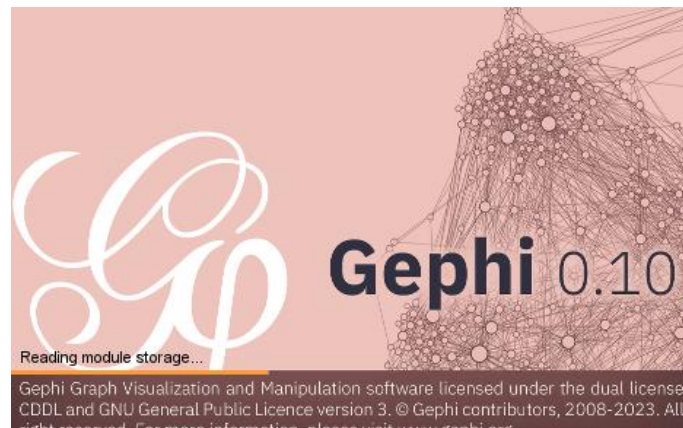
- **Polarity (\$P\$)**: A floating-point value ranging precisely from **-1.0 to +1.0**. A score of -1.0 represents maximum negative sentiment, 0.0 represents perfectly neutral text, and +1.0 represents maximum positive praise.
- **Subjectivity (\$S\$)**: A metric ranging from **0.0 to 1.0** that determines how factual or opinionated the text is. A score closer to 0.0 indicates highly objective, fact-driven language, while scores closer to 1.0 signal purely subjective, emotion-driven opinions.

2. Real-World Problem Statement

- **Problem**: An e-commerce platform or corporate brand launches a major consumer product but lacks the staff resources to manually read thousands of daily online customer submissions. Without automation, negative product defects or brand public relations crises go unnoticed.
- **Objective**: Scrape product review columns from public websites and pipeline the text data through a sentiment engine to track brand satisfaction, providing immediate alerts for negative customer complaints.

LAB WORK

App



1. Upload dataset

Spreadsheet (CSV)...

Steps

1. General CSV options
2. Import settings

General CSV options (1 of 2)

CSV file to import:

C:\Users\hp\OneDrive\Desktop\dm\retail_sales_dataset.csv

Separator: Comma Import as: Adjacency list Charset: UTF-8

Preview:

Transaction...	Date	Customer ID	Gender	Age	Product Ca...	Quantity
1	2023-11-24	CUST001	Male	34	Beauty	3
2	2023-02-27	CUST002	Female	26	Clothing	2
3	2023-01-13	CUST003	Male	50	Electronics	1
4	2023-05-21	CUST004	Male	37	Clothing	1
5	2023-05-06	CUST005	Male	30	Beauty	2
6	2023-04-25	CUST006	Female	45	Beauty	1

< Back Next > Finish Cancel Help

Import report

Source: retail_sales_dataset.csv

Issues Report

Nodes Issues

Parallel edges detected, remember to choose a merge strategy INFO

Graph Type: Directed

of Nodes: 2362

of Edges: 8008

Dynamic Graph: no

Dynamic Attributes: no

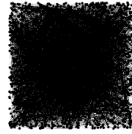
Multi Graph: no

New workspace (selected)

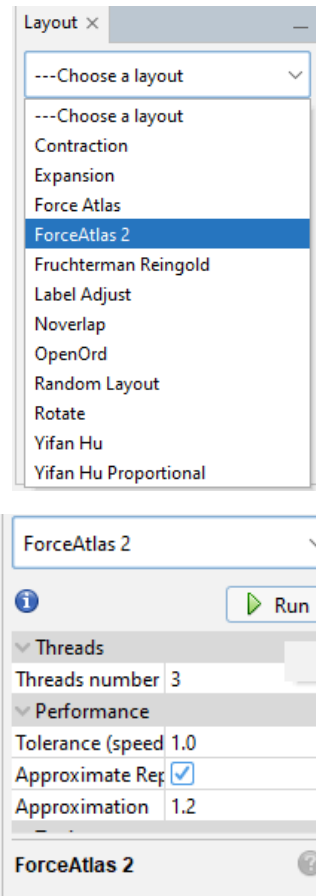
Append to existing workspace

More options...

OK Cancel



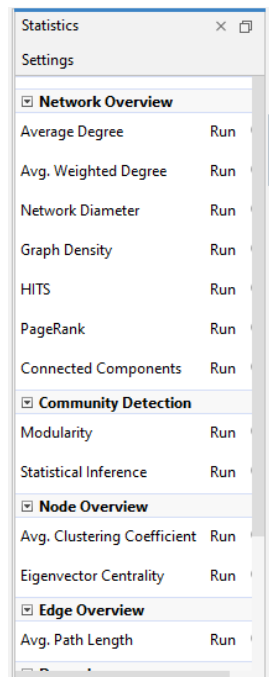
2. Choose layout



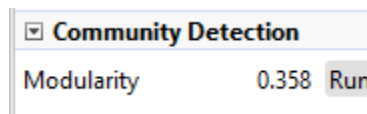
3. After running



4. Select statistics



5. Modularity



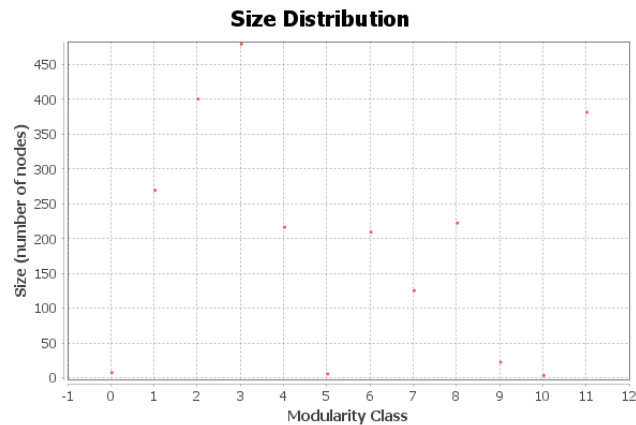
Modularity Report

Parameters:

Randomize: On
Use edge weights: On
Resolution: 1.0

Results:

Modularity: 0.358
Modularity with resolution: 0.358
Number of Communities: 12



Algorithm:

Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, Etienne Lefebvre, *Fast unfolding of communities in large networks*, in Journal of Statistical Mechanics: Theory and Experiment 2008 (10), P1000

Resolution:

R. Lambiotte, J.-C. Delvenne, M. Barahona *Laplacian Dynamics and Multiscale Modular Structure in Networks* 2009

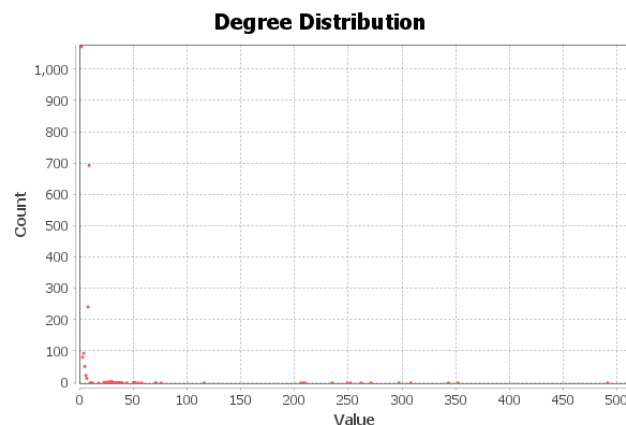
6. Average degree

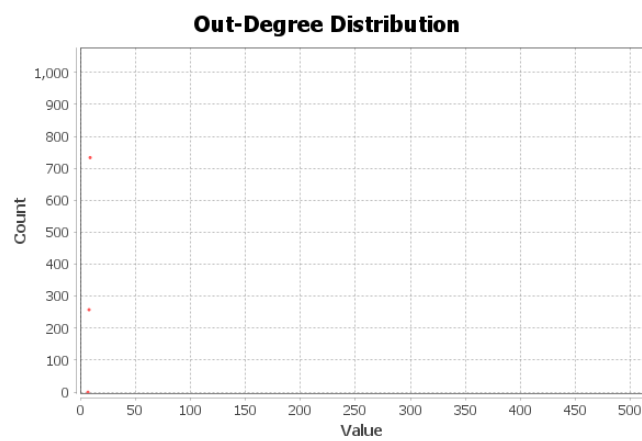
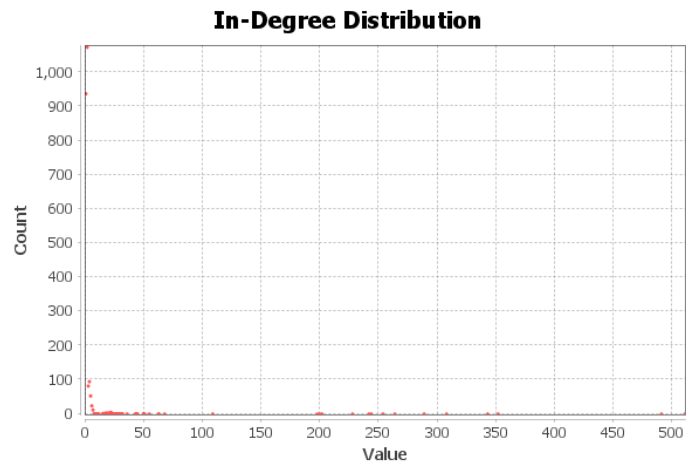
Average Degree 3.277 Run

Degree Report

Results:

Average Degree: 3.277





7. Graph density

Graph Density 0.001

Graph Density Report

Parameters:

Network Interpretation: directed

Results:

Density: 0.001

8. Edge overview

Avg. Path Length 4.137

Graph Distance Report

Parameters:

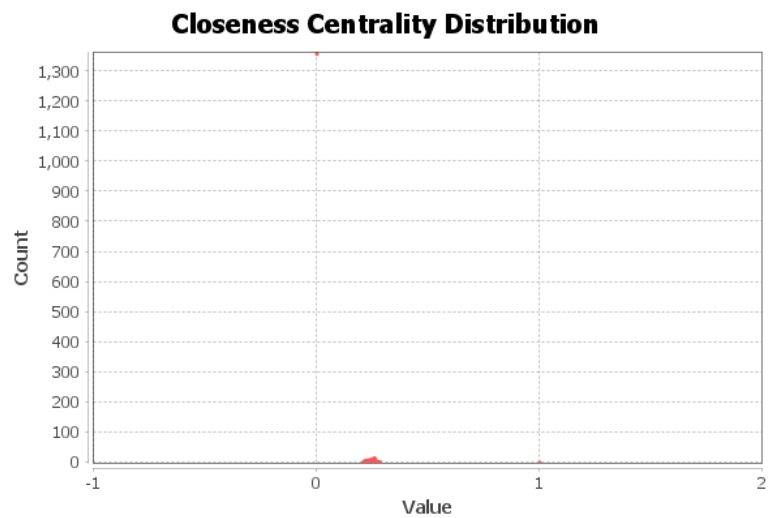
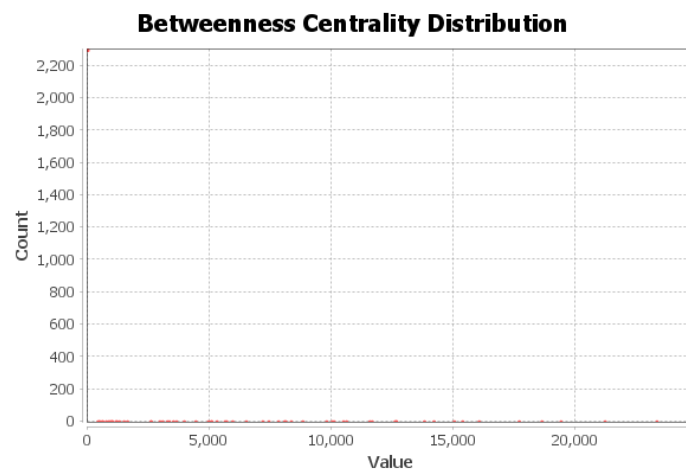
Network Interpretation: directed

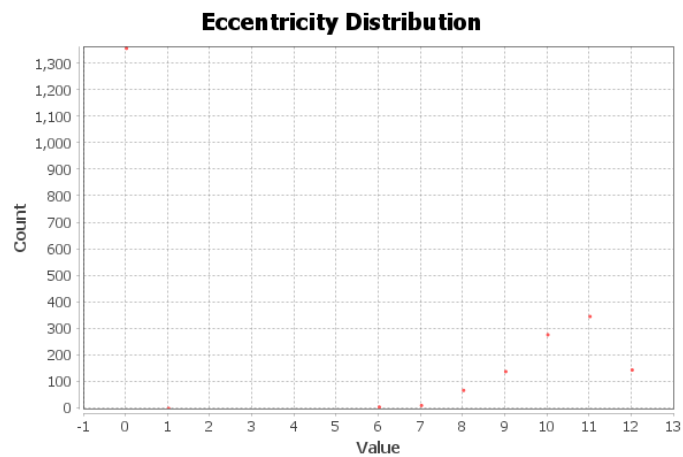
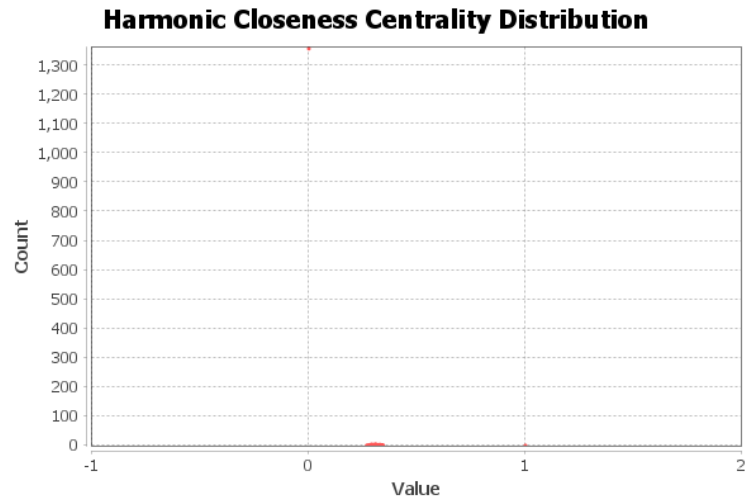
Results:

Diameter: 12

Radius: 0

Average Path length: 4.136677325899202

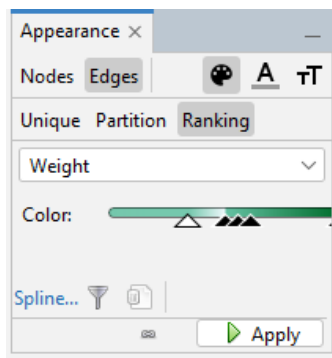




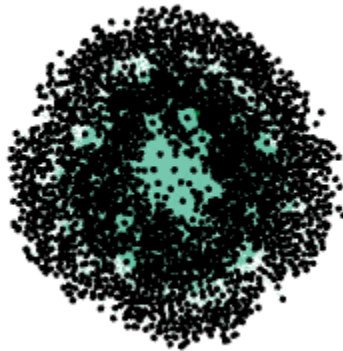
Algorithm:

Ulrik Brandes, *A Faster Algorithm for Betweenness Centrality*, in Journal of Mathematical Sociology 25(2):163-177, (2001)

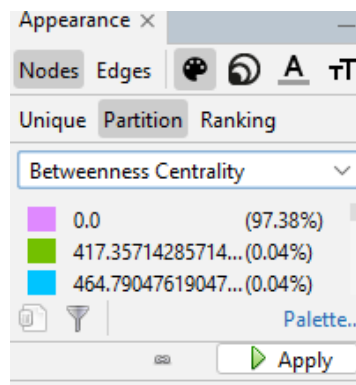
9. From edges



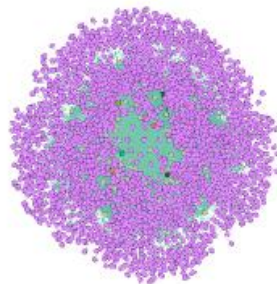
After applying

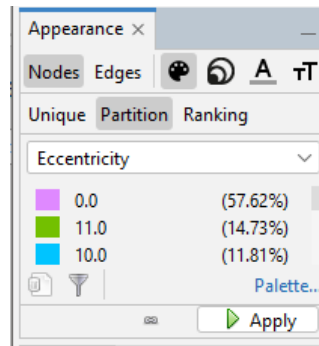


10. From nodes

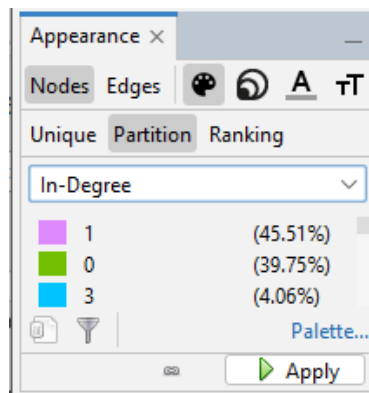
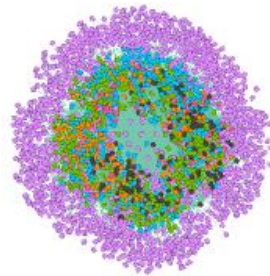


After applying

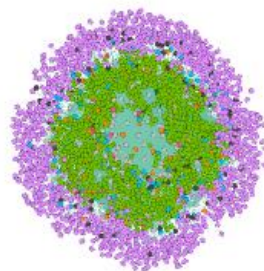


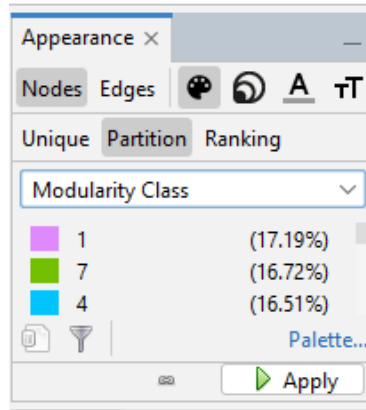


After applying

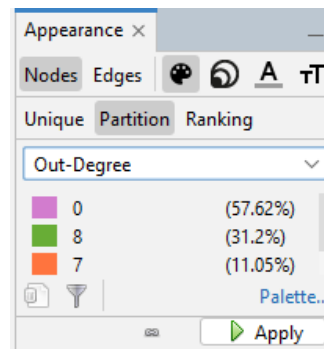


After applying

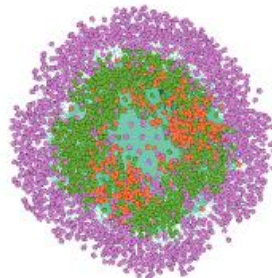


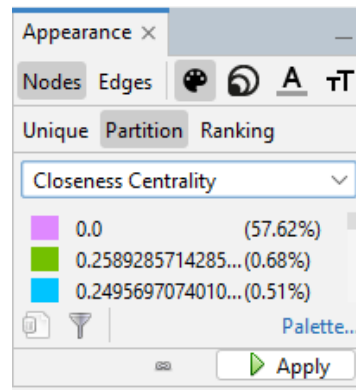


After applying

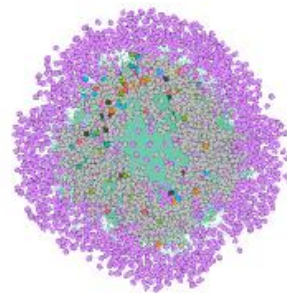


After applying

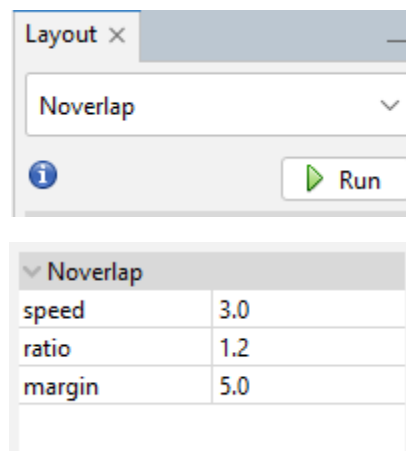




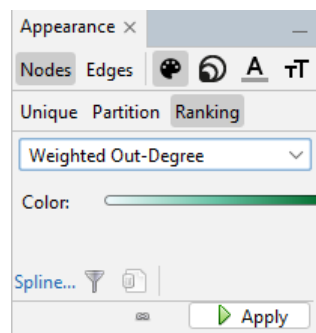
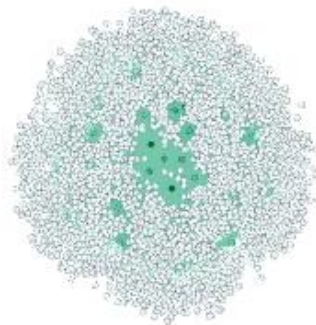
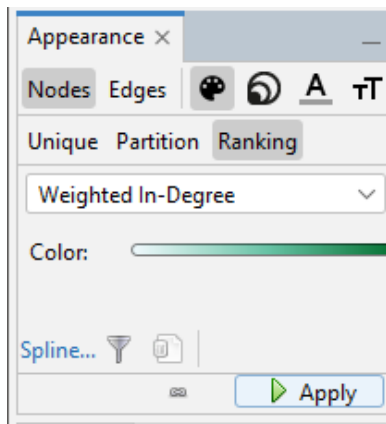
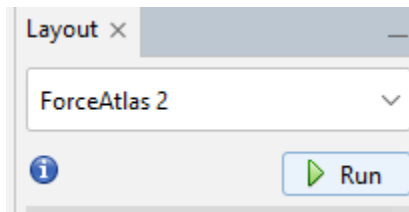
After applying

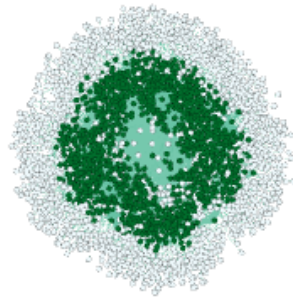


11. Change layout

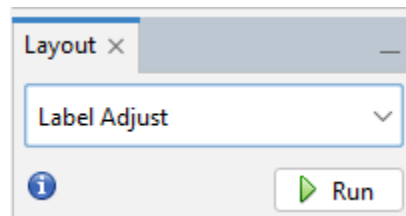


12. No change, back to atlas2

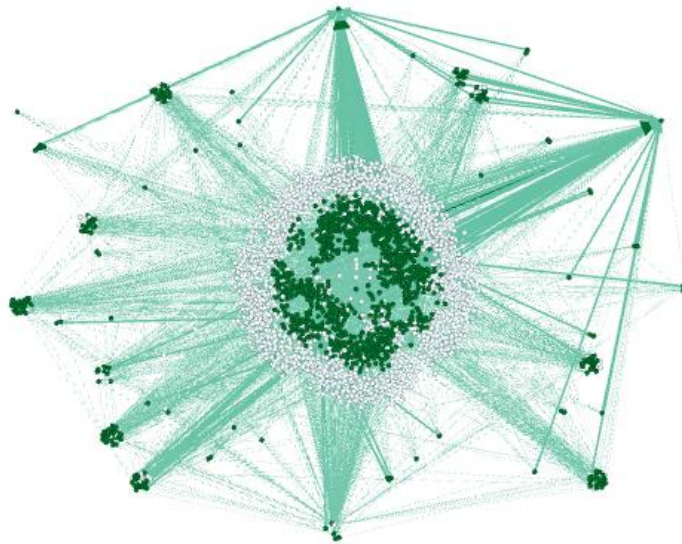




13. Change layout



Result



LAB 15

FINAL CAPSTONE

1. Introduction & Theoretical Framework

1.1 The CRISP-DM Framework

The Final Capstone Project requires the execution of a complete, non-linear **CRISP-DM (Cross-Industry Standard Process for Data Mining)** cycle. Rather than focusing on a single isolated algorithm, this lab focuses on translating an ambiguous business problem into a deployed data framework.

The project explicitly dictates moving through the 6 standard phases:

1. **Business Understanding:** Defining organizational objectives, identifying operational bottlenecks, and formatting the target analytical goals.
2. **Data Understanding:** Ingesting raw industry data pipelines, exploring attribute distributions, and checking initial data quality metrics.
3. **Data Preparation:** Executing rigorous cleaning, handling outliers, transforming columns via normalization, and engineering features.
4. **Modeling:** Applying and tuning **8 distinct machine learning models** side-by-side to solve the objective.
5. **Evaluation:** Benchmarking models against testing partitions to pick the champion model.
6. **Deployment:** Formatting the analytical findings into concrete, actionable strategic business recommendations.

2. Real-World Problem Statement

- **Problem:** A enterprise institution (e.g., Telecom, Banking, Retail, or Healthcare) suffers from operational inefficiencies, high customer attrition, or unidentified risk exposures. Relying on simple historical hunches or single-model algorithms leads to fragile market execution and missed revenue targets.
- **Objective:** Design, optimize, and evaluate an end-to-end data mining pipeline. By training and pitting 8 different classification/regression systems against each other, students must establish a robust "champion model" that provides specific, highly reliable business logic for executive strategy deployment.

3. Target Dataset Options

Your manual leaves this open to **Any Industry Dataset** chosen by the analytics group based on project focus. Typical selections from previous tasks include:

- *Telecom Churn Dataset* (Classification)
- *Supermarket / Retail Sales Dataset* (Regression/Analysis)
- *Credit Card Fraud & Risk Dataset* (Imbalanced Classification)

LAB WORK

Dataset

 RADHESHYAM KOLLIPARA · UPDATED 3 YEARS AGO

271

Code

Download

Bank Customer Churn

Bank Customer Data for Customer Churn



Data Card

Code (170)

Discussion (13)

Suggestions (0)

About Dataset

RowNumber—corresponds to the record (row) number and has no effect on the output.
CustomerId—contains random values and has no effect on customer leaving the bank.
Surname—the surname of a customer has no impact on their decision to leave the bank.
CreditScore—can have an effect on customer churn, since a customer with a higher credit score is less likely to leave the bank.
Geography—a customer's location can affect their decision to leave the bank.
Gender—it's interesting to explore whether gender plays a role in a customer leaving the bank.

Usability ⓘ
10.00

License
Other (specified in description)

Expected update frequency
Annually

Taqs

1. Import libraries

```
[5] ✓ Os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc
```

2. Load dataset

```
[6] ✓ Os
df = pd.read_csv('Customer-Churn-Records.csv')
print("--- Initial Dataset Info ---")
print(df.info())
print("\nFirst 5 Rows:")
print(df.head())

--- Initial Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   RowNumber             10000 non-null  int64
 1   CustomerId            10000 non-null  int64
 2   Surname               10000 non-null  object
 3   CreditScore           10000 non-null  int64
 4   Geography             10000 non-null  object
 5   Gender               10000 non-null  object
 6   Age                  10000 non-null  int64
 7   Tenure               10000 non-null  int64
 8   Balance              10000 non-null  float64
 9   NumOfProducts        10000 non-null  int64
10   HasCrCard            10000 non-null  int64
11   IsActiveMember       10000 non-null  int64
12   EstimatedSalary      10000 non-null  float64
13   Exited               10000 non-null  int64
14   Complain             10000 non-null  int64
15   Satisfaction Score   10000 non-null  int64
16   Card Type            10000 non-null  object
17   Point Earned         10000 non-null  int64
dtypes: float64(2), int64(12), object(4)
memory usage: 1.4+ MB
None
```

First 5 Rows:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited	Complain	Satisfaction	Score	Card Type	\
0	101348.88	1	1		2	DIAMOND	
1	112542.58	0	1		3	DIAMOND	
2	113931.57	1	1		3	DIAMOND	
3	93826.63	0	0		5	GOLD	
4	79084.10	0	0		5	GOLD	

	Point Earned
0	464
1	456
2	377
3	350
4	425

3. Data pre=processing and cleaning

```
[7]
✓ Os
# Drop irrelevant columns that don't help prediction
cols_to_drop = ['RowNumber', 'CustomerId', 'Surname']
df = df.drop(columns=[col for col in cols_to_drop if col in df.columns])

# Handle Missing Values (if any)
df = df.dropna()

# Encode Categorical Variables (Text to Numbers)
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Define Features (X) and Target (y)
target_col = 'Exited'
X = df.drop(columns=[target_col])
y = df[target_col]

# Split data into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling (Standardization)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nPreprocessing Complete. Data split into 80% Train and 20% Test.")

Preprocessing Complete. Data split into 80% Train and 20% Test.
```

4. Implementing 8 diff models

```
[8] ✓ 6s
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier

# Initialize the 8 models
models = {
    "1. Logistic Regression": LogisticRegression(),
    "2. Decision Tree": DecisionTreeClassifier(random_state=42),
    "3. Naive Bayes": GaussianNB(),
    "4. K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
    "5. SVM (Linear Kernel)": SVC(kernel='linear', probability=True, random_state=42),
    "6. SVM (RBF Kernel)": SVC(kernel='rbf', probability=True, random_state=42),
    "7. Random Forest": RandomForestClassifier(random_state=42),
    "8. Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

# Dictionary to hold our final evaluation metrics
results_dict = {}

print("\n--- Training and Evaluating 8 Models ---")
for name, model in models.items():
    # Train the model
    model.fit(X_train_scaled, y_train)

    # Predict on test set
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1] # Needed for ROC-AUC
```

```
6s
# Calculate performance metrics (KPIs)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, y_prob)

# Store metrics in the dictionary
results_dict[name] = {
    "Accuracy (%)": round(accuracy * 100, 2),
    "Precision (%)": round(precision * 100, 2),
    "Recall (%)": round(recall * 100, 2),
    "F1-Score (%)": round(f1 * 100, 2),
    "ROC-AUC": round(roc_auc, 3)
}

print(f"Finished training: {name}")
```

```
...
--- Training and Evaluating 8 Models ---
Finished training: 1. Logistic Regression
Finished training: 2. Decision Tree
Finished training: 3. Naive Bayes
Finished training: 4. K-Nearest Neighbors
Finished training: 5. SVM (Linear Kernel)
Finished training: 6. SVM (RBF Kernel)
Finished training: 7. Random Forest
Finished training: 8. Gradient Boosting
```

5. Final capstone performance

```
[9] # Convert results into a structured DataFrame
✓ Os comparison_df = pd.DataFrame.from_dict(results_dict, orient='index')

print("\n=====")
print("          FINAL MODEL COMPARISON TABLE          ")
print("=====")
print(comparison_df.to_string())
print("=====\\n")

# Find the best model based on Accuracy
best_model_name = comparison_df['Accuracy (%)'].idxmax()
best_accuracy = comparison_df.loc[best_model_name, 'Accuracy (%)']

=====
          FINAL MODEL COMPARISON TABLE
=====

```

	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	ROC-AUC
1. Logistic Regression	99.9	99.75	99.75	99.75	0.999
2. Decision Tree	99.7	99.24	99.24	99.24	0.995
3. Naïve Bayes	99.9	99.75	99.75	99.75	0.999
4. K-Nearest Neighbors	99.7	99.74	98.73	99.23	0.999
5. SVM (Linear Kernel)	99.9	99.75	99.75	99.75	0.999
6. SVM (RBF Kernel)	99.9	99.75	99.75	99.75	0.998
7. Random Forest	99.9	99.75	99.75	99.75	1.000
8. Gradient Boosting	99.8	99.24	99.75	99.49	0.999

6. Business interpretation and recommendation

```
[10]
✓ Os print("=====")
print("          BUSINESS RECOMMENDATION          ")
print("=====")
print(f"Based on the performance metrics, the winning model is: **{best_model_name}**")
print(f"It achieved a peak baseline predictive Accuracy of {best_accuracy}%.")
print("\nStrategic Action Plan for the Bank:")
print(f"1. Deploy the {best_model_name} model within the bank's CRM system.")
print("2. Flag high-risk accounts automatically before they actively close their accounts.")
print("3. Target flagged accounts with proactive loyalty incentives (e.g., lower interest rates, zero fees).")
print("4. This data-driven framework will help mitigate customer churn and preserve annual banking revenue.")
print("=====")

=====
          BUSINESS RECOMMENDATION
=====
Based on the performance metrics, the winning model is: **1. Logistic Regression**
It achieved a peak baseline predictive Accuracy of 99.9%.

Strategic Action Plan for the Bank:
1. Deploy the 1. Logistic Regression model within the bank's CRM system.
2. Flag high-risk accounts automatically before they actively close their accounts.
3. Target flagged accounts with proactive loyalty incentives (e.g., lower interest rates, zero fees).
4. This data-driven framework will help mitigate customer churn and preserve annual banking revenue.
=====
```